

RAPPORT DE STAGE

Partie Théorique

Partie Pratique

Première partie — Mise en œuvre avec interface graphique

Deuxième partie — Mise en œuvre en ligne de commande

Thème :

Authentification AAA Basée sur Serveur

Réalisé par : SAHRAOUI TUDERT & CHERIFFI SAAD

Domaine du stage : Sécurité Réseau & Infrastructure

Environnement : Ubuntu 24.04 · OpenLDAP · FreeRADIUS · dom10.dz

TABLE DES MATIÈRES

PARTIE THEORIQUE

I.	Contexte général de l'entreprise et évolution du réseau.....	3
II.	Limites de la gestion locale des accès	5
III.	Problématique	6
IV.	Notion fondamentale : l'authentification réseau	6
V.	Types d'authentification dans les réseaux d'entreprise	8
VI.	Le modèle AAA : Authentication, Authorization, Accounting	10
VII.	Les protocoles d'authentification réseau.....	12
VIII.	Le standard IEEE 802.1X et le protocole EAP	14
IX.	Les principales solutions AAA du marché.....	15
X.	Architecture globale d'une solution AAA	16
XI.	De la théorie à la pratique : présentation du laboratoire.....	18
	Conclusion.....	20

PARTIE PRATIQUE

Partie I — AAA Local sur Routeur Cisco IOS	22
Partie II — AAA Basé sur Serveur : Configuration du Routeur Cisco.....	25
Partie III — Travail Pratique : Déploiement du Serveur AAA	28
Bibliographie.....	53

I. Contexte général de l'entreprise et évolution du réseau

Dans le paysage numérique actuel, le système d'information constitue le cœur opérationnel de toute entreprise. Qu'il s'agisse d'accéder aux applications métier, de communiquer via des plateformes collaboratives, de consulter des bases de données ou de se connecter au réseau distant depuis un site secondaire, les ressources réseau sont sollicitées en permanence et par un nombre croissant d'acteurs.

Cette réalité impose aux responsables informatiques de concevoir des infrastructures à la fois robustes, évolutives et sécurisées. Or, cette sécurisation ne commence pas uniquement au niveau des pare-feux ou des antivirus : elle commence dès le premier point de contact entre un utilisateur et le réseau, c'est-à-dire au niveau de l'authentification.

1.1 L'infrastructure réseau à ses débuts

Au commencement d'une activité, l'infrastructure réseau est généralement simple et de faible envergure. Elle se compose d'un nombre limité d'équipements : quelques commutateurs (switches), un ou deux routeurs, des points d'accès Wi-Fi, et un petit nombre de postes de travail.

Dans ce contexte, l'administration réseau reste accessible et directe. L'administrateur configure chaque équipement individuellement, en se connectant localement ou via une console physique. Les comptes utilisateurs sont créés directement sur l'équipement lui-même :

- un compte administrateur est défini sur chaque switch ou routeur ;
- les mots de passe sont enregistrés localement dans la configuration ;
- les droits d'accès sont gérés équipement par équipement, sans logique centralisée.

Cette approche est parfaitement viable à petite échelle. Elle ne nécessite aucun serveur dédié et peut être mise en place rapidement. Cependant, elle porte en elle les germes de ses propres limitations, qui se manifesteront inévitablement avec la croissance de l'organisation.

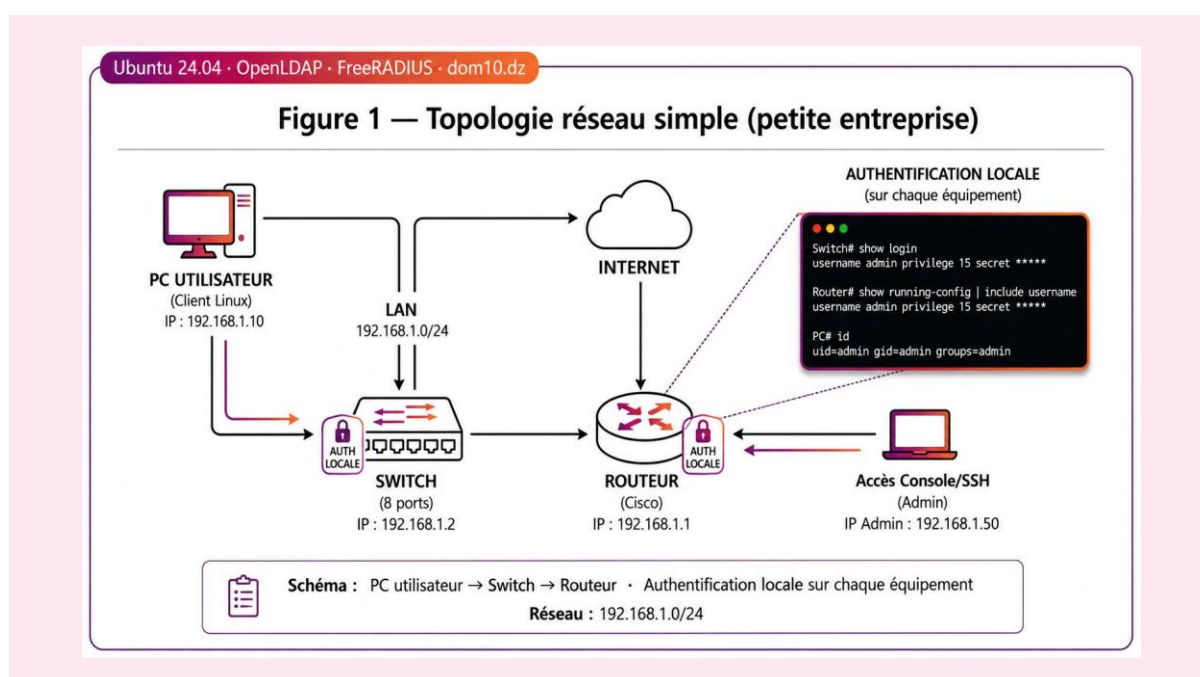


Figure 1 — Topologie réseau simple (petite entreprise)

Schéma : PC utilisateur → Switch → Routeur · Authentification locale sur chaque équipement

1.2 L'évolution vers une infrastructure complexe

Avec le développement de l'entreprise, le réseau suit naturellement la même trajectoire de croissance. De nouveaux départements sont créés, des bâtiments supplémentaires sont intégrés, des accès distants sont rendus nécessaires pour les télétravailleurs ou les sites secondaires. Le nombre d'équipements réseau se multiplie, et avec lui, le nombre d'utilisateurs à gérer.

Cette transition peut amener l'infrastructure à intégrer simultanément :

- plusieurs dizaines ou centaines d'équipements réseau actifs ;
- des connexions Wi-Fi sécurisées pour les employés et les visiteurs ;
- des tunnels VPN pour les accès distants ;
- des ressources Cloud hybrides nécessitant une authentification fédérée ;
- des systèmes de supervision et de journalisation centralisés.

Cette complexité croissante expose immédiatement les limites du modèle d'authentification locale, lequel, pensé pour quelques équipements, ne peut pas passer à l'échelle de manière fiable.

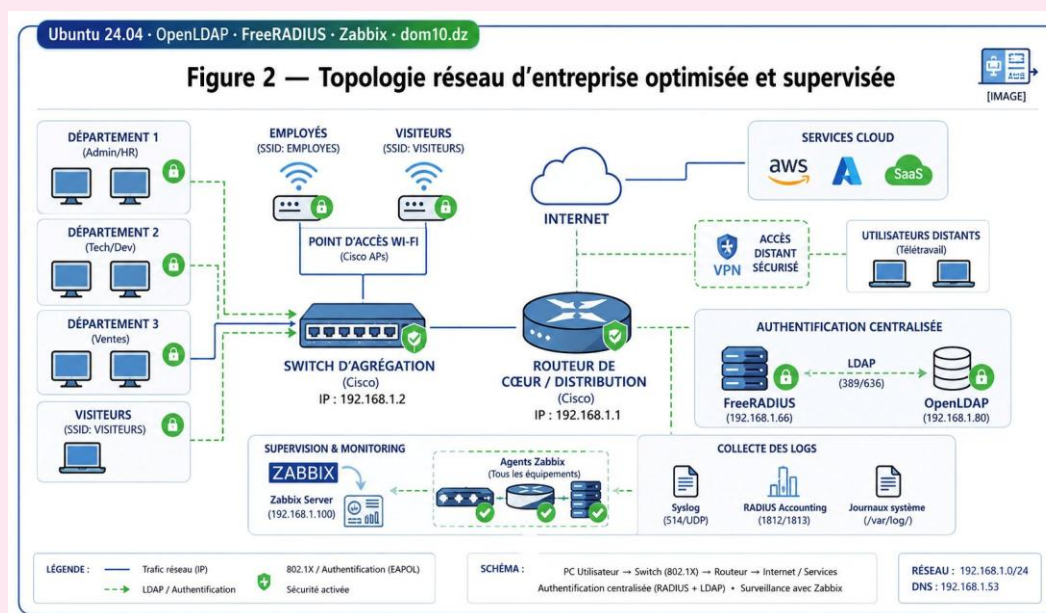
**Figure 2 — Topologie réseau d'entreprise complexe**

Schéma : plusieurs départements, VPN, Wi-Fi, accès distants, Cloud — besoin de centralisation

II. Limites de la gestion locale des accès

La gestion locale des accès consiste à maintenir, sur chaque équipement réseau, une base d'utilisateurs propre. Bien que fonctionnelle dans un environnement restreint, cette approche révèle rapidement des insuffisances majeures à mesure que l'infrastructure grandit.

2.1 Problèmes d'administration

Imaginons une entreprise disposant de cinquante switches et vingt routeurs. Si l'administrateur réseau doit créer, modifier ou supprimer un compte utilisateur, il devra répéter la même opération soixante-dix fois — une fois sur chaque équipement. Cette redondance de tâches est non seulement chronophage, mais elle constitue également une source d'erreurs humaines significative.

Concrètement, les problèmes rencontrés sont les suivants :

- Duplication des configurations : chaque équipement possède sa propre liste d'utilisateurs, ses propres mots de passe et ses propres niveaux de privilège ;
- Inconsistances : un compte peut être mis à jour sur certains équipements mais oublié sur d'autres, créant des états incohérents difficiles à détecter ;
- Gestion des départs : lorsqu'un administrateur quitte l'entreprise, la révocation de ses droits doit être effectuée manuellement sur l'ensemble du parc — une tâche fastidieuse, propice aux oublis.

2.2 Problèmes de sécurité

Au-delà de la complexité administrative, la gestion locale pose un problème fondamental de sécurité : l'absence de traçabilité centralisée. Dans ce modèle, il est extrêmement difficile de répondre aux questions élémentaires d'un audit de sécurité :

- Quel utilisateur s'est connecté à tel équipement, et à quelle heure ?
- Quelles commandes ont été exécutées lors de cette session ?
- Y a-t-il eu des tentatives d'accès non autorisées ?

Sans journalisation centralisée, chaque équipement conserve ses propres logs (quand il en génère), dispersés et difficiles à corréler. Une investigation post-incident devient alors un exercice laborieux, voire impossible dans les délais requis par la gestion de crise.

⚠ Synthèse des limites de la gestion locale

- Administration répétitive et non centralisée
- Risque élevé d'erreurs humaines lors des mises à jour
- Impossibilité de garantir la cohérence des droits d'accès
- Absence de traçabilité centralisée des connexions
- Gestion des départs d'employés complexe et risquée
- Faible évolutivité : le modèle ne passe pas à l'échelle

III. Problématique

Face à l'inadéquation du modèle local avec les exigences des infrastructures modernes, une question stratégique s'impose naturellement à toute organisation en phase de croissance :

Comment gérer de manière centralisée, sécurisée et évolutive l'authentification des utilisateurs sur les équipements réseau, tout en garantissant un contrôle précis des droits, une traçabilité complète des actions et une administration simplifiée ?

Cette problématique est au cœur du stage présenté dans ce rapport. Pour y répondre, il est nécessaire d'explorer le concept d'authentification réseau, les protocoles qui le sous-tendent, et le modèle AAA qui en constitue la référence architecturale moderne. La solution mise en œuvre pratiquement — un serveur FreeRADIUS couplé à un annuaire OpenLDAP sur le domaine dom10.dz — vient apporter une réponse concrète et opérationnelle à cette problématique.

IV. Notion fondamentale : l'authentification réseau

4.1 Définition

L'authentification est le processus par lequel un système informatique vérifie qu'une entité — utilisateur, machine ou service — est bien celle qu'elle prétend être, avant de lui accorder l'accès à une ressource. C'est le gardien numérique de l'infrastructure : sans lui, aucune distinction ne peut être faite entre un employé légitime et un intrus.

L'authentification répond à la question fondamentale : « Qui êtes-vous, et pouvez-vous le prouver ? »

Elle repose sur la présentation de preuves d'identité, qui se regroupent généralement en trois catégories :

- Ce que l'on sait : un mot de passe, un code PIN, une réponse à une question secrète ;
- Ce que l'on possède : une carte à puce, un token physique, un certificat numérique ;
- Ce que l'on est : une empreinte biométrique (digitale, faciale, iris).

Dans les réseaux d'entreprise, la combinaison de plusieurs de ces facteurs — ce qu'on appelle l'authentification multi-facteurs (MFA) — est de plus en plus répandue pour renforcer le niveau de sécurité.

4.2 Importance dans l'entreprise

Dans un environnement professionnel, les ressources à protéger sont nombreuses et variées. Un mécanisme d'authentification solide permet de :

- contrôler qui peut accéder aux équipements réseau (routeurs, switches, points d'accès) ;
- protéger les serveurs hébergeant les données sensibles de l'entreprise ;
- sécuriser les connexions distantes via VPN ;
- gérer les accès aux applications internes et aux plateformes Cloud ;
- limiter la surface d'attaque en cas de compromission de compte.

Sans authentification, n'importe quel utilisateur connecté au réseau pourrait potentiellement accéder à l'ensemble des ressources de l'entreprise, ouvrant la voie à des fuites de données, des sabotages ou des attaques internes. L'authentification est donc le premier rempart, mais elle ne suffit pas seule : elle s'inscrit dans un cadre plus large, celui du modèle AAA.

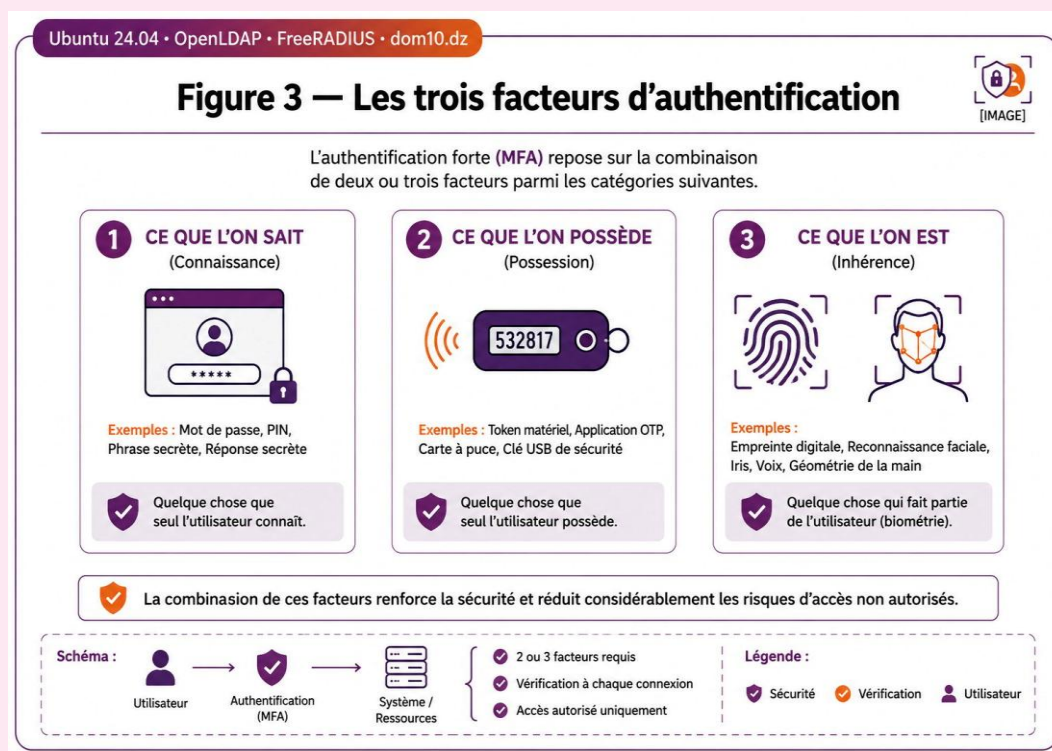


Figure 3 — Les trois facteurs d'authentification

Schéma : Ce que l'on sait (mot de passe) / Ce que l'on possède (token) / Ce que l'on est (biométrie)

V. Types d'authentification dans les réseaux d'entreprise

5.1 Authentification locale

Dans ce modèle, chaque équipement réseau maintient sa propre base de données d'utilisateurs. Un switch Cisco, par exemple, peut stocker localement un nom d'utilisateur et un mot de passe pour permettre l'accès à son interface d'administration.

Avantages	Inconvénients
Simplicité de mise en œuvre	Aucune centralisation
Fonctionne sans serveur externe	Duplication des comptes sur chaque équipement
Utile pour les petits réseaux	Administration très lourde à grande échelle
Résistance aux pannes réseau	Traçabilité quasi inexistante
Idéal comme solution de secours	Risque élevé d'incohérences et d'oublis

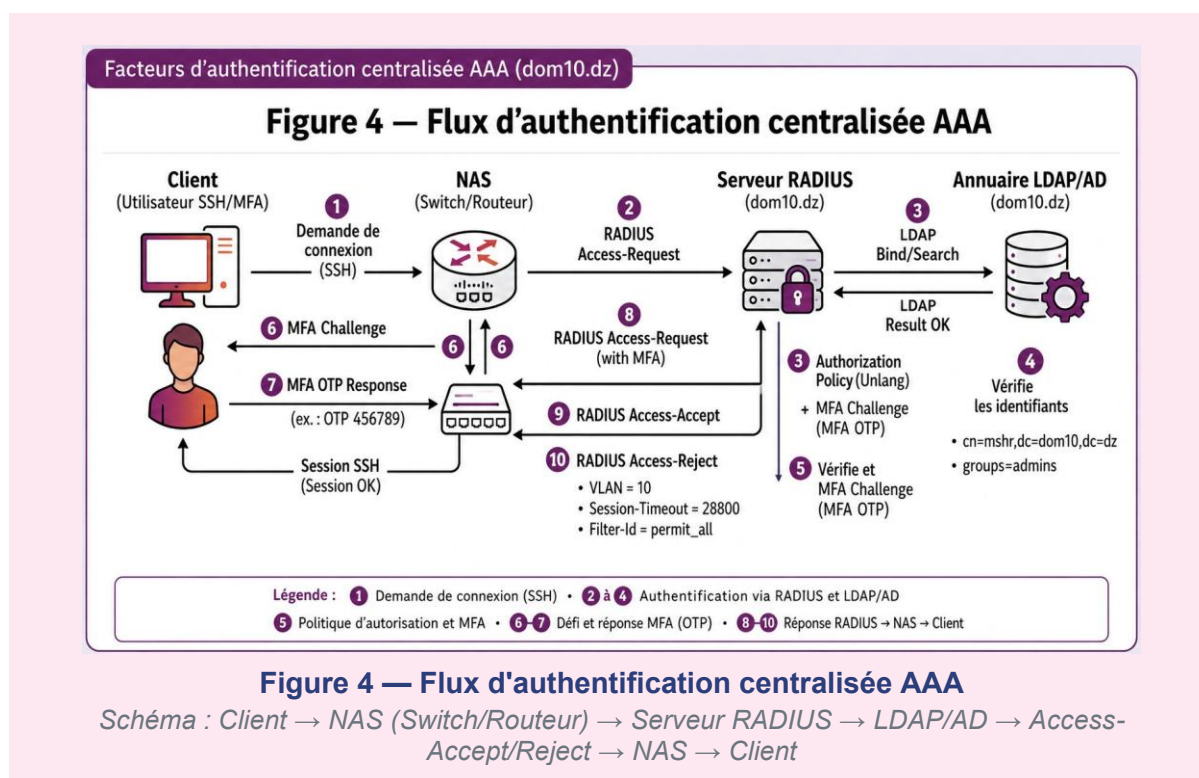
Dans le cadre de notre stage, l'authentification locale est conservée comme mécanisme de fallback : un compte secours est configuré sur le routeur Cisco pour garantir l'accès en cas d'indisponibilité du serveur RADIUS. Cette pratique est fortement recommandée en production.

5.2 Authentification centralisée basée sur serveur

Pour dépasser les limites de la gestion locale, les entreprises adoptent un modèle centralisé dans lequel les équipements réseau délèguent la vérification des identités à un serveur spécialisé. Ce serveur, appelé serveur AAA, devient le point de décision unique pour tous les accès réseau.

Le flux d'authentification centralisée se déroule de la manière suivante :

- L'utilisateur tente de se connecter à un équipement réseau (routeur, switch, point d'accès).
- L'équipement (appelé NAS — Network Access Server) intercepte la demande et la transmet au serveur AAA.
- Le serveur AAA consulte sa base de données (locale ou externe, comme LDAP/Active Directory).
- Le serveur renvoie une décision : Access-Accept (accès autorisé) ou Access-Reject (accès refusé).
- L'équipement applique la décision et ouvre ou ferme la session.

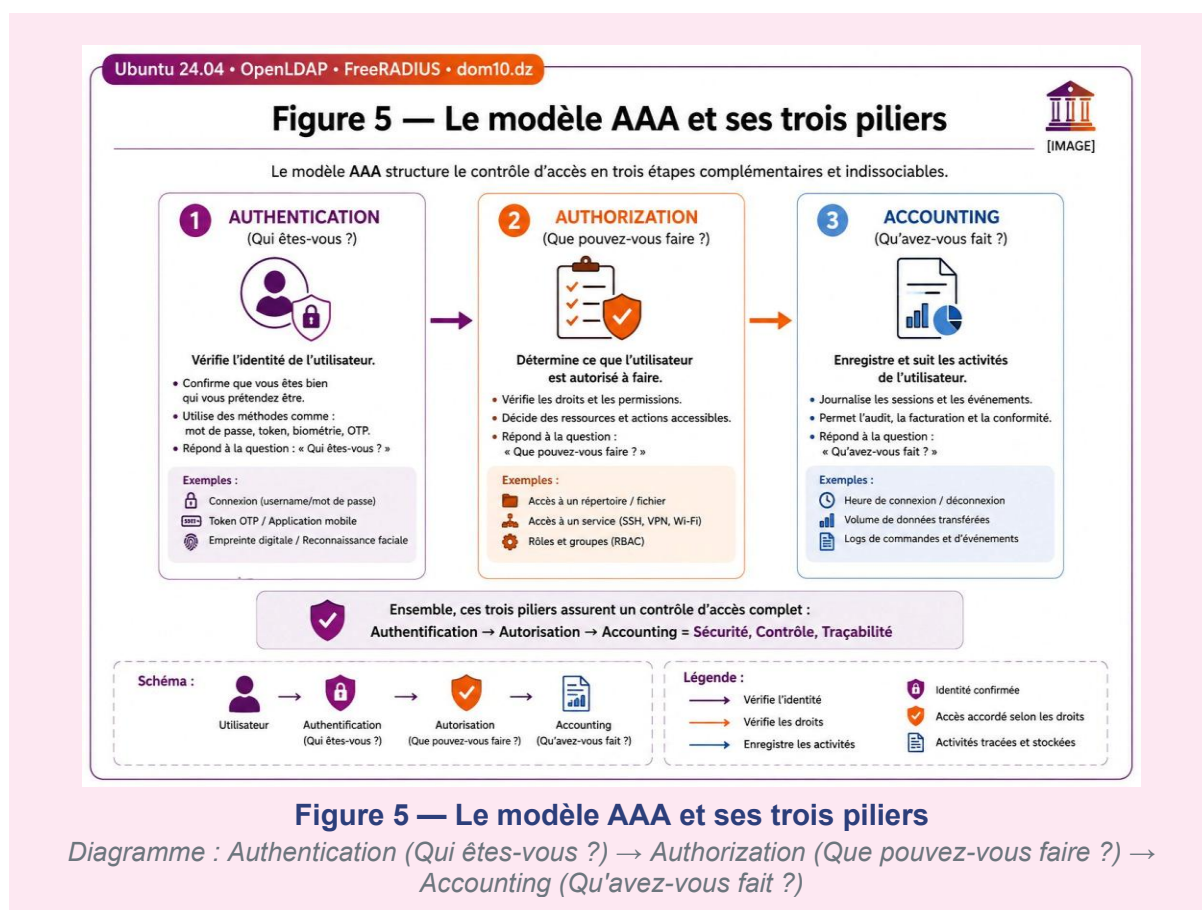


☑ Avantages clés de l'authentification centralisée

- Un seul point d'administration pour tous les comptes utilisateurs
- Cohérence garantie des droits et politiques de sécurité
- Journalisation centralisée de toutes les connexions
- Modification immédiate et globale (création, suppression, modification de compte)
- Scalabilité : le modèle supporte des milliers d'équipements et d'utilisateurs

VI. Le modèle AAA : Authentication, Authorization, Accounting

Le modèle AAA est le cadre architectural qui structure la gestion des accès réseau dans les infrastructures modernes. Il décompose le contrôle d'accès en trois fonctions distinctes et complémentaires, dont l'interaction garantit une sécurité complète et une administration maîtrisée.



6.1 Authentication — L'identification

La première fonction du modèle AAA est l'authentification. Elle répond à la question : « Cet utilisateur est-il réellement celui qu'il prétend être ? »

Le processus implique la présentation de preuves d'identité (identifiant et mot de passe, certificat numérique, etc.) que le serveur AAA valide en les comparant avec les informations stockées dans sa base de données — dans notre cas pratique, l'annuaire OpenLDAP du domaine dom10.dz.

Si la vérification réussit, l'utilisateur est authentifié. Le serveur envoie un message Access-Accept à l'équipement réseau, qui autorise l'établissement de la session.

6.2 Authorization — Le contrôle des droits

Une fois l'identité confirmée, la deuxième fonction entre en jeu : l'autorisation. Elle répond à la question : « Que cet utilisateur est-il autorisé à faire ? »

L'autorisation permet de définir, pour chaque utilisateur ou groupe, un ensemble précis de droits et de restrictions :

- les commandes pouvant être exécutées sur un équipement (niveau de privilège sur Cisco : privilege level 1 à 15) ;
- les ressources réseau accessibles (certains VLANs, certains serveurs) ;
- la durée maximale des sessions ;
- les plages horaires d'accès autorisées.

Dans notre mise en œuvre pratique, l'annuaire LDAP organise les utilisateurs en groupes distincts (admins, users, invites), ce qui permet d'appliquer des politiques d'autorisation différenciées selon le profil de l'utilisateur connecté.

6.3 Accounting — La traçabilité

La troisième et dernière fonction du modèle AAA est l'accounting (ou journalisation). Elle répond à la question : « Qu'a fait cet utilisateur pendant sa session ? »

L'accounting enregistre automatiquement, dans des fichiers de journalisation centralisés, l'ensemble des événements liés aux connexions :

- l'heure de début et de fin de chaque session ;
- l'identité de l'utilisateur et l'équipement accédé ;
- la durée totale de la session ;
- les commandes exécutées (avec TACACS+) ;
- les tentatives d'accès refusées.

Ces données sont précieuses pour plusieurs usages : audits de sécurité, investigations post-incident, conformité réglementaire, et analyse du comportement des utilisateurs. Dans notre laboratoire pratique, les logs FreeRADIUS sont stockés sous `/var/log/freeradius/` et peuvent être analysés pour vérifier chaque tentative de connexion.

Fonction AAA	Question clé	Action du serveur	Bénéfice principal
Authentication	Qui êtes-vous ?	Vérifie l'identité dans LDAP	Sécurisation des accès
Authorization	Que pouvez-vous faire ?	Applique les politiques de droits	Contrôle granulaire
Accounting	Qu'avez-vous fait ?	Enregistre les événements	Traçabilité et audit

VII. Les protocoles d'authentification réseau

Pour implémenter le modèle AAA, plusieurs protocoles standardisés ont été développés. Chacun répond à des cas d'usage spécifiques, avec des caractéristiques distinctes en termes de sécurité, de chiffrement et de compatibilité.

7.1 RADIUS — Remote Authentication Dial-In User Service

RADIUS est le protocole AAA le plus répandu dans les réseaux d'entreprise. Défini à l'origine pour les connexions distantes (dial-up), il est aujourd'hui utilisé dans une grande variété de contextes : authentification Wi-Fi, accès VPN, contrôle d'accès aux équipements réseau.

RADIUS fonctionne sur le modèle client-serveur. L'équipement réseau (NAS) agit comme client RADIUS : il reçoit les demandes de connexion et les transmet au serveur RADIUS pour décision. La communication entre le NAS et le serveur utilise le protocole UDP sur le port 1812 pour l'authentification et le port 1813 pour l'accounting.

Caractéristiques techniques de RADIUS

- Protocole : UDP (ports 1812/auth, 1813/accounting)
- Chiffrement : seul le mot de passe est chiffré (avec MD5 + clé partagée)
- Fonctions AAA : toutes trois intégrées dans un seul paquet
- Compatibilité : universelle (tous les équipements réseau)
- Implémentation open source : FreeRADIUS (utilisé dans ce stage)

Dans notre stage, FreeRADIUS est utilisé comme serveur RADIUS. Il reçoit les requêtes des équipements réseau (le routeur Cisco simulé en laboratoire), interroge l'annuaire LDAP pour vérifier les identités, et renvoie les décisions d'accès. La clé partagée (testing123) sécurise la communication entre le NAS et le serveur.

7.2 TACACS+ — Terminal Access Controller Access-Control System Plus

TACACS+ est un protocole développé par Cisco, particulièrement adapté à l'administration des équipements réseau (routeurs, switches). Sa différence fondamentale avec RADIUS réside dans l'architecture de ses fonctions AAA : là où RADIUS les regroupe dans un même échange, TACACS+ les sépare complètement, offrant ainsi un contrôle beaucoup plus granulaire.

TACACS+ chiffre l'intégralité du payload de chaque paquet (et non seulement le mot de passe), ce qui en fait une solution plus sécurisée pour les accès administratifs sensibles.

Critère	RADIUS	TACACS+
Protocole transport	UDP	TCP
Port	1812 / 1813	49
Chiffrement	Mot de passe uniquement	Payload complet
Séparation AAA	Non (intégré)	Oui (indépendant)
Usage principal	Wi-Fi, VPN, accès réseau	Administration équipements
Support constructeurs	Universel	Principalement Cisco

7.3 LDAP — Lightweight Directory Access Protocol

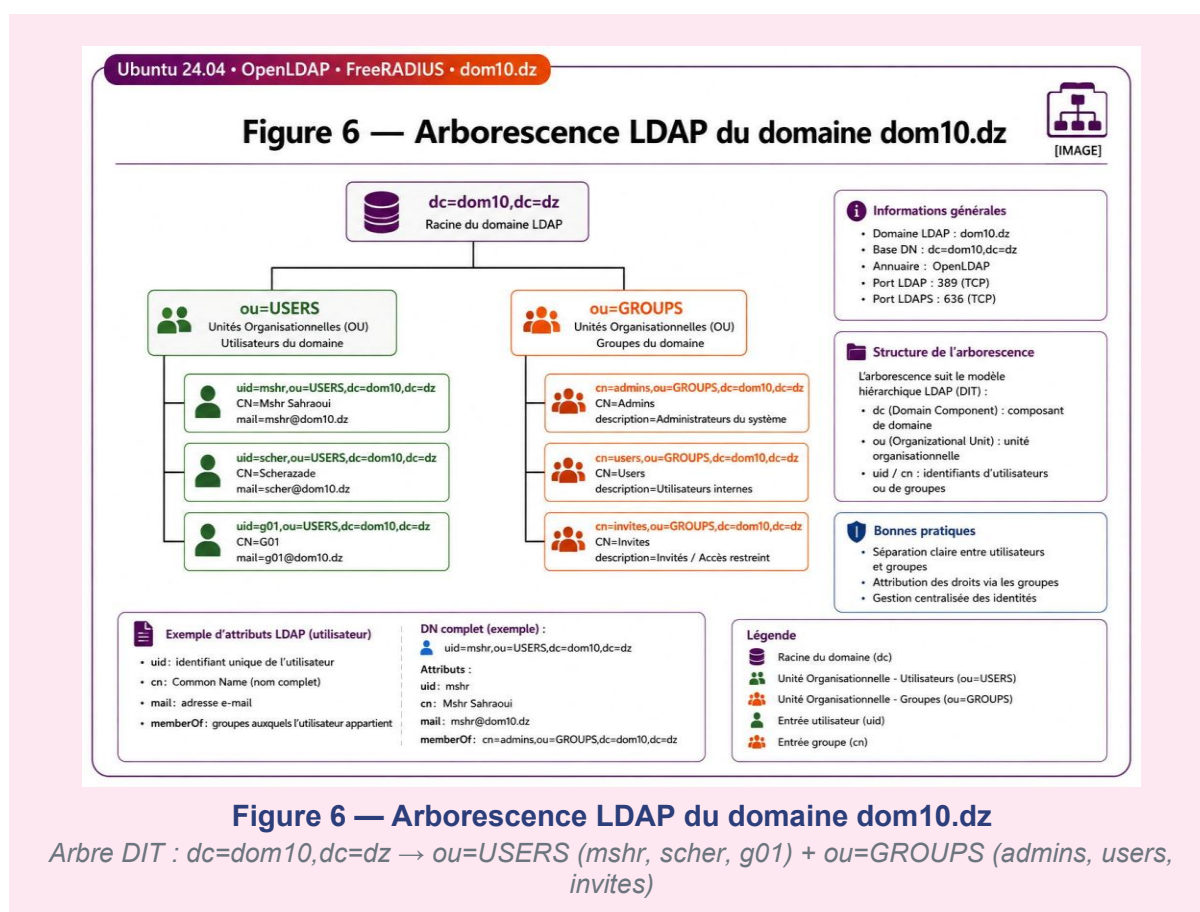
LDAP n'est pas un protocole AAA à proprement parler, mais il joue un rôle fondamental dans les architectures modernes de contrôle d'accès. Il s'agit d'un protocole d'accès à un service

d'annuaire, c'est-à-dire une base de données hiérarchique qui centralise les informations sur les utilisateurs, les groupes et les ressources de l'organisation.

Dans une architecture AAA, LDAP sert de backend d'authentification : le serveur RADIUS reçoit une demande d'accès, puis interroge l'annuaire LDAP pour vérifier si l'utilisateur existe et si son mot de passe est correct. C'est exactement le rôle joué par OpenLDAP dans notre mise en œuvre pratique sur le domaine dom10.dz.

L'arborescence LDAP est organisée de manière hiérarchique, en suivant le modèle DIT (Directory Information Tree) :

- dc=dom10,dc=dz : la racine du domaine ;
- ou=USERS,dc=dom10,dc=dz : l'unité organisationnelle regroupant les comptes utilisateurs ;
- ou=GROUPS,dc=dom10,dc=dz : l'unité regroupant les groupes (admins, users, invites) ;
- cn=admin,dc=dom10,dc=dz : le compte administrateur LDAP.



VIII. Le standard IEEE 802.1X et le protocole EAP

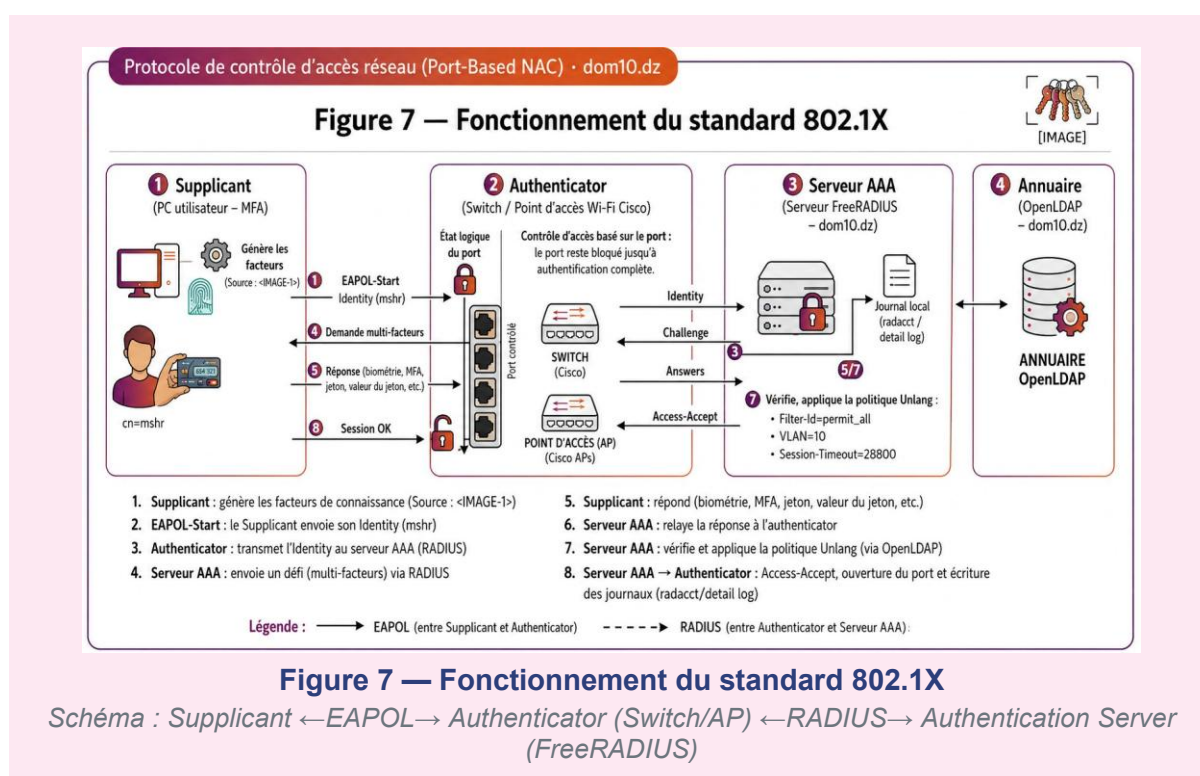
Au-delà des protocoles RADIUS et TACACS+, les réseaux modernes s'appuient sur un standard fondamental pour contrôler l'accès au niveau de la couche liaison : le standard IEEE 802.1X. Ce standard est particulièrement pertinent dans notre stage, car il est directement mis en œuvre dans la partie pratique pour sécuriser l'accès au réseau Wi-Fi.

8.1 Architecture du standard 802.1X

Le standard 802.1X repose sur trois acteurs qui interagissent selon un protocole précis avant qu'un accès réseau soit accordé :

Rôle	Entité réseau	Fonction
Supplicant	Poste de travail / périphérique	Fournit les informations d'identité (login, certificat)
Authenticator	Switch ou point d'accès Wi-Fi	Intercepte le trafic et le redirige vers le serveur AAA
Authentication Server	Serveur RADIUS (FreeRADIUS)	Vérifie l'identité et renvoie la décision d'accès

Le principe du 802.1X est simple mais puissant : le port réseau (physique ou Wi-Fi) reste bloqué jusqu'à ce que l'identité du périphérique connecté soit vérifiée et validée par le serveur AAA. Cette approche dite de « contrôle d'accès basé sur le port » (Port-Based Network Access Control) garantit qu'aucun équipement non autorisé ne peut accéder au réseau, même s'il est physiquement connecté.



8.2 Le protocole EAP — Extensible Authentication Protocol

EAP est le protocole de transport utilisé à l'intérieur du framework 802.1X pour acheminer les messages d'authentification entre le supplicant et le serveur d'authentification. Sa grande force est son extensibilité : il ne définit pas une méthode d'authentification spécifique, mais un cadre dans lequel de nombreuses méthodes peuvent être implémentées.

Les méthodes EAP les plus courantes dans les environnements d'entreprise sont :

- EAP-TLS (Transport Layer Security) : utilise des certificats numériques côté client et serveur. C'est la méthode la plus robuste, car elle ne repose pas sur des mots de passe. En contrepartie, elle nécessite une infrastructure PKI (Public Key Infrastructure) complète.
- EAP-PEAP (Protected EAP) avec MSCHAPv2 : crée d'abord un tunnel TLS pour protéger l'échange, puis utilise MSCHAPv2 pour transmettre le nom d'utilisateur et le mot de passe dans ce tunnel sécurisé. Très répandu dans les environnements Windows/Active Directory. C'est la méthode configurée dans la partie pratique de ce stage.
- EAP-TTLS (Tunneled TLS) : similaire à PEAP, mais plus flexible sur le choix du protocole d'authentification interne.

Dans notre mise en œuvre pratique, EAP-PEAP est configuré sur le serveur FreeRADIUS pour permettre l'authentification 802.1X des clients Wi-Fi du domaine dom10.dz. Le module eap de FreeRADIUS gère l'établissement du tunnel TLS et la vérification des credentials via LDAP.

IX. Les principales solutions AAA du marché

Plusieurs solutions logicielles permettent d'implémenter le modèle AAA dans les entreprises. Leur choix dépend de la taille de l'organisation, du budget disponible, de l'environnement existant et des besoins en termes de fonctionnalités.

9.1 Cisco Identity Services Engine (ISE)

Cisco ISE est la solution AAA de référence pour les grandes entreprises disposant d'un parc réseau Cisco. Elle offre des fonctionnalités avancées de contrôle d'accès réseau (NAC — Network Access Control), de segmentation, de profilage des périphériques et d'intégration avec des systèmes de détection des menaces.

Cisco ISE supporte nativement les protocoles RADIUS et TACACS+, et s'intègre avec Active Directory pour la gestion des identités. Sa richesse fonctionnelle s'accompagne cependant d'une complexité de déploiement et d'un coût de licence élevés, ce qui la réserve aux grandes infrastructures.

9.2 FreeRADIUS — La solution open source de référence

FreeRADIUS est le serveur RADIUS open source le plus déployé au monde. Gratuit, flexible et hautement configurable, il est utilisé aussi bien dans des laboratoires pédagogiques que dans des infrastructures de production à grande échelle.

FreeRADIUS supporte une large gamme de méthodes d'authentification (PAP, CHAP, MSCHAPv2, EAP-TLS, EAP-PEAP, etc.) et peut s'interfacer avec de nombreuses sources d'identités : LDAP, Active Directory, SQL, Kerberos, etc. C'est la solution choisie pour notre stage, car elle offre un excellent rapport fonctionnalité/complexité pour un environnement d'apprentissage.

FreeRADIUS dans notre stage

- Version : FreeRADIUS 3.x sur Ubuntu 24.04
- Module LDAP : freeradius-ldap (liaison avec OpenLDAP)
- Module EAP : gestion de EAP-PEAP/MSCHAPv2 pour le Wi-Fi

- Clients configurés : Switch_Principal (192.168.1.10)
- Logs : /var/log/freeradius/ (accounting et authentification)

9.3 Network Policy Server (NPS) — Windows Server

NPS est le serveur RADIUS intégré à Windows Server. Il s'intègre nativement avec Active Directory, ce qui le rend particulièrement adapté aux environnements Microsoft. Il permet de définir des politiques de connexion réseau complexes, en combinant des critères d'identité, d'appartenance à des groupes, de santé du poste de travail (Network Access Protection) et de plage horaire.

NPS est souvent déployé dans les entreprises qui ont déjà standardisé leur infrastructure autour de Windows Server et d'Active Directory, évitant ainsi l'installation et la maintenance d'une solution tierce.

Solution	Type	Protocoles	Intégration	Usage idéal
Cisco ISE	Commercial	RADIUS, TACACS+	Active Directory, Cisco	Grande entreprise Cisco
FreeRADIUS	Open Source	RADIUS, EAP	LDAP, AD, SQL	Laboratoire, PME, Production
NPS (Windows)	Intégré Windows	RADIUS	Active Directory natif	Environnement Microsoft

X. Architecture globale d'une solution AAA

Pour récapituler l'ensemble des concepts abordés, il est utile de visualiser l'architecture complète d'une solution AAA telle qu'elle est déployée dans les entreprises modernes et telle qu'elle a été mise en œuvre dans notre stage.

10.1 Les composants de l'architecture

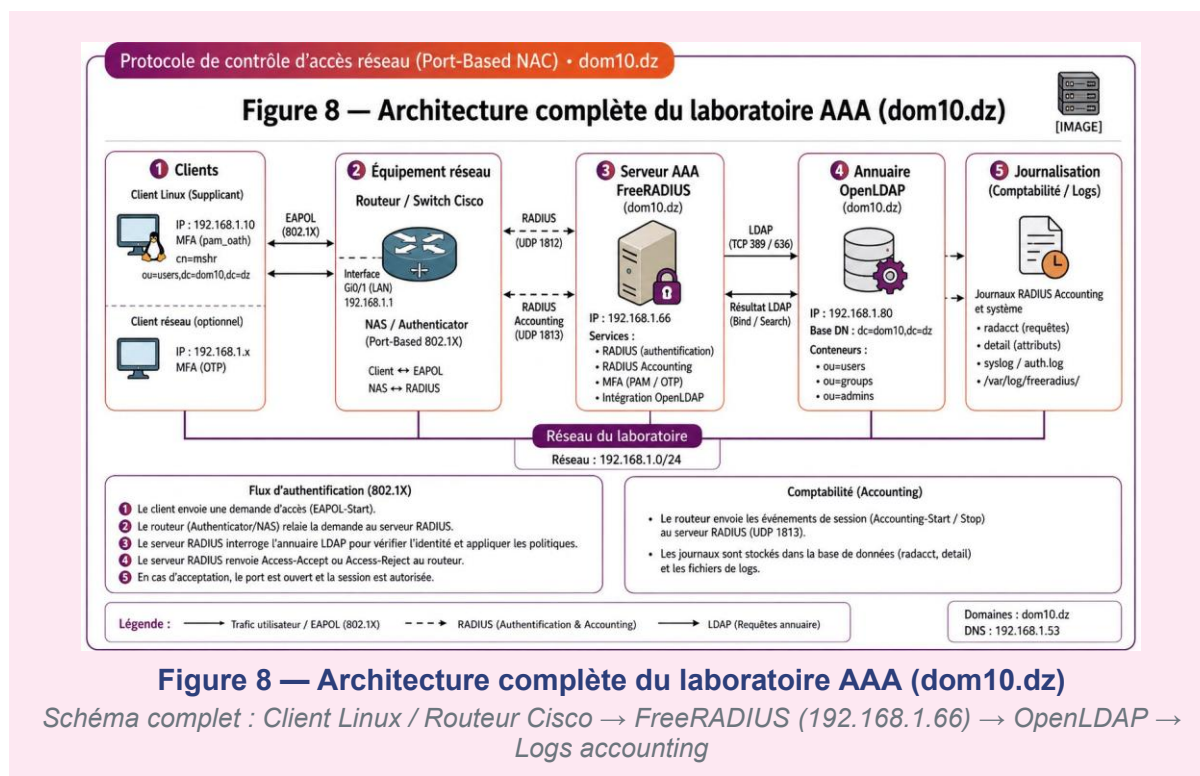
Une architecture AAA complète implique les composants suivants :

- Le client (utilisateur ou machine) : le supplicant qui demande l'accès au réseau. Dans notre stage, il s'agit d'un poste Ubuntu ou du routeur Cisco.
- Le NAS — Network Access Server : l'équipement réseau (switch, routeur, point d'accès) qui intercepte la demande et la transmet au serveur AAA. Dans notre laboratoire, il s'agit du routeur Cisco configuré pour déléguer l'authentification à FreeRADIUS.
- Le serveur AAA (FreeRADIUS) : le cœur du système. Il reçoit les requêtes, applique les politiques, consulte l'annuaire et renvoie les décisions.
- L'annuaire LDAP (OpenLDAP / Active Directory) : la base de données centralisée des identités. Dans notre stage, il s'agit de l'annuaire dom10.dz hébergé sur le serveur 192.168.1.66.
- Le système de journalisation : les logs générés par le serveur AAA, qui enregistrent toutes les authentifications, refus d'accès et durées de session.

10.2 Le flux complet d'une connexion

Le déroulement complet d'une authentification AAA réussie dans notre environnement suit les étapes suivantes :

Étape	Action	Acteur	Résultat
1	L'utilisateur tente de se connecter au routeur via SSH	Client (mshr)	Demande de connexion
2	Le routeur intercepte la demande et envoie une requête RADIUS	NAS (Cisco Router)	Access-Request envoyé
3	FreeRADIUS reçoit la requête et interroge l'annuaire LDAP	Serveur FreeRADIUS	Requête LDAP vers dom10.dz
4	OpenLDAP vérifie les credentials de mshr dans dc=dom10,dc=dz	OpenLDAP (192.168.1.66)	Validation ou rejet
5	FreeRADIUS renvoie Access-Accept au routeur	Serveur FreeRADIUS	Accès autorisé
6	La session est établie et enregistrée dans les logs d'accounting	FreeRADIUS + NAS	Session tracée



XI. De la théorie à la pratique : présentation du laboratoire

Les concepts présentés dans cette partie théorique trouvent une application directe dans la mise en œuvre pratique réalisée lors de ce stage. Chaque notion abordée — le modèle AAA, les protocoles RADIUS et LDAP, le standard 802.1X, les méthodes EAP — correspond à une étape concrète de configuration dans le laboratoire.

11.1 L'environnement du laboratoire

Le laboratoire s'appuie sur les composants suivants, déployés sur une infrastructure virtuelle Ubuntu 24.04 :

Composant	Technologie	Adresse IP / Paramètre	Rôle
Serveur AAA	FreeRADIUS 3.x	192.168.1.66	Traitement des requêtes RADIUS
Annuaire LDAP	OpenLDAP + phpLDAPadmin	192.168.1.66 (port 389)	Base centralisée des identités
Domaine	dom10.dz	dc=dom10,dc=dz	Espace de noms LDAP
Client SSSD	Ubuntu 24.04	192.168.1.X	Authentification système via LDAP
Équipement réseau	Cisco IOS (simulé)	192.168.1.10 (NAS)	Délégation AAA vers FreeRADIUS

11.2 Les utilisateurs du domaine

Trois utilisateurs ont été créés dans l'annuaire LDAP, appartenant à des groupes distincts afin de tester les mécanismes d'autorisation différenciée :

Utilisateur	Groupe LDAP	Rôle simulé	Niveau d'accès
mshr (Melissa)	admins	Administrateur réseau	Accès complet (privilege 15)
scher	users	Utilisateur standard	Accès limité aux commandes utilisateur
g01	invites	Invité externe	Accès restreint, lecture seule

Cette organisation en groupes illustre concrètement la fonction Authorization du modèle AAA : une fois authentifié, chaque utilisateur se voit appliquer des droits différents selon son appartenance groupée, sans aucune configuration individuelle sur les équipements réseau eux-mêmes.

11.3 Correspondance théorie-pratique

Chaque section de la partie pratique correspond directement à l'un des concepts théoriques présentés dans ce rapport :

Concept théorique	Mise en œuvre pratique
Authentification centralisée AAA	FreeRADIUS recevant les requêtes RADIUS du routeur Cisco
Annuaire LDAP comme backend	OpenLDAP dom10.dz vérifiant les credentials de mshr/scher/g01
Protocole RADIUS (port 1812/1813)	Communication Switch_Principal ↔ FreeRADIUS avec clé testing123
Autorisation par groupe	Groupes admins/users/invites dans LDAP + politiques FreeRADIUS
Accounting et journalisation	Logs /var/log/freeradius/radius.log et fichiers d'accounting
Standard 802.1X + EAP-PEAP	Configuration Wi-Fi avec certificat TLS sur FreeRADIUS
Fallback local (secours)	Compte admin_local sur le routeur Cisco en cas de panne serveur

🎯 Objectifs de la partie pratique

- Déployer un serveur FreeRADIUS opérationnel sur Ubuntu 24.04
- Configurer l'annuaire OpenLDAP avec le domaine dom10.dz et ses utilisateurs
- Établir la liaison FreeRADIUS ↔ LDAP pour l'authentification déléguée
- Configurer un routeur Cisco pour déléguer son authentification au serveur RADIUS
- Mettre en place le 802.1X avec EAP-PEAP pour l'accès Wi-Fi sécurisé
- Vérifier la journalisation (accounting) de toutes les connexions

Conclusion

Ce rapport théorique a présenté, de manière progressive et structurée, l'ensemble des concepts qui fondent la compréhension et la mise en œuvre d'une infrastructure d'authentification AAA basée sur serveur.

Nous avons vu comment la croissance d'une entreprise rend inévitable le passage d'une gestion locale des accès, simple mais non scalable, à un modèle centralisé reposant sur le triptyque Authentication-Authorization-Accounting. Ce modèle, incarné par des protocoles éprouvés comme RADIUS et supporté par des solutions comme FreeRADIUS et OpenLDAP, offre à l'entreprise les garanties de sécurité, de traçabilité et d'évolutivité qu'elle est en droit d'attendre de son infrastructure réseau.

Le standard IEEE 802.1X et le protocole EAP viennent compléter ce dispositif en étendant le contrôle d'accès aux couches les plus fondamentales du réseau : les ports physiques des switches et les accès Wi-Fi, garantissant qu'aucun équipement non authentifié ne peut accéder au réseau de l'entreprise.

La partie pratique qui suit ce rapport illustre concrètement l'ensemble de ces principes, en déployant un laboratoire complet sur Ubuntu 24.04 avec FreeRADIUS, OpenLDAP et un routeur Cisco simulé. Chaque configuration réalisée trouve sa justification dans les concepts théoriques présentés ici, assurant ainsi la cohérence et la complémentarité des deux parties de ce travail de stage.

— Fin de la partie théorique —

RAPPORT DE STAGE

Authentification AAA Basée sur Serveur

PARTIE II — MISE EN ŒUVRE PRATIQUE

Environnement : Ubuntu 24.04 · FreeRADIUS 3.x · OpenLDAP · dom10.dz
Serveur AAA : 192.168.1.66 · Client : cnode1 · Secret : testing123

Note : Cette partie pratique est également disponible sous forme de site web interactif à titre de complément visuel : <https://tudertshr.github.io/AAA-freeRadius/>

Introduction — Du général au spécifique

La partie théorique a posé les fondements conceptuels de l'infrastructure AAA : le modèle Authentication-Authorization-Accounting, les protocoles RADIUS et LDAP, le standard IEEE 802.1X et le protocole EAP. Cette partie pratique suit une progression pédagogique délibérée : partir du plus simple pour aller vers le plus sophistiqué, en trois étapes successives.

La première étape illustre l'AAA dans son expression la plus directe : un routeur Cisco configuré en mode local, sans aucun serveur externe. C'est le point de départ universel, compréhensible et reproductible par tout administrateur réseau. La deuxième étape franchit le cap de la centralisation : le même routeur délègue désormais ses décisions d'accès à un serveur RADIUS externe, illustrant concrètement le gain en scalabilité et en cohérence. La troisième étape présente le travail pratique réalisé dans le cadre de ce stage : le déploiement complet d'une infrastructure AAA serveur sur Ubuntu 24.04, avec FreeRADIUS, OpenLDAP, SSSD et Zabbix.

Étape	Contexte	Technologie	Objectif
1	AAA local sur routeur	Cisco IOS — base de données locale	Comprendre la mécanique de base AAA sans serveur
2	AAA basé sur serveur — côté routeur	Cisco IOS + FreeRADIUS externe	Déléguer l'authentification à un serveur RADIUS
3	Travail Pratique personnel	Ubuntu 24.04 — FreeRADIUS + OpenLDAP + SSSD	Déployer et opérer un serveur AAA complet

Partie I — AAA Local sur Routeur Cisco IOS

Avant d'aborder la centralisation, il est essentiel de comprendre le modèle d'authentification le plus simple : celui où le routeur lui-même héberge la base de données des utilisateurs. C'est le point de départ de toute infrastructure réseau, et c'est sur cette base que le modèle AAA s'est construit historiquement.

I.1 Contexte et justification de l'AAA local

Sur un routeur Cisco, l'accès à l'interface d'administration (CLI) peut être protégé par deux méthodes : le mot de passe de console/VTY simple, ou une authentification AAA avec base locale. L'AAA local est recommandé car il permet d'avoir des comptes nominatifs (non anonymes), des niveaux de privilège différenciés, et une traçabilité par utilisateur. Sans AAA activé, tous les accès partagent le même mot de passe, sans distinction d'identité.

i Vocabulaire Cisco IOS

• aaa new-model	: active le framework AAA sur le routeur (obligatoire avant toute règle AAA)
• username ... privilege	: crée un utilisateur local avec un niveau de privilège (1 à 15)
• aaa authentication login	: définit la méthode d'authentification pour les connexions
• line vty / line con 0	: configure les lignes d'accès (SSH/Telnet et console physique)
• login authentication	: applique une liste d'authentification AAA à une ligne

I.2 Configuration étape par étape

Étape 1 — Activer le framework AAA

La commande `aaa new-model` est le point d'entrée obligatoire. Elle désactive immédiatement l'ancienne méthode d'authentification (login local sans AAA) et place le routeur sous le contrôle du framework AAA. À partir de ce moment, toute connexion doit correspondre à une règle AAA définie.

```
Router> enable
Router# configure terminal
Router(config)# aaa new-model
! ↑ Active le framework AAA — OBLIGATOIRE en premier
```

Étape 2 — Créer les utilisateurs locaux

On crée trois comptes avec des niveaux de privilège différents, reflétant les trois rôles que nous retrouverons ensuite dans l'annuaire LDAP : administrateur (niveau 15 = accès complet), utilisateur standard (niveau 5 = accès limité) et invité (niveau 1 = lecture seule).

```
! Administrateur — accès total (privilege 15)
Router(config)# username mshr privilege 15 secret X123456x

! Utilisateur standard — accès limité (privilege 5)
Router(config)# username scher privilege 5 secret X123456x

! Invité — lecture seule (privilege 1)
```

```
Router(config)# username g01 privilege 1 secret X123456x

! Vérifier la liste des utilisateurs locaux
Router# show running-config | include username
```

Étape 3 — Définir les méthodes d'authentification AAA

Une liste d'authentification AAA nomme la séquence de méthodes que le routeur consulte. Le mot-clé local indique d'utiliser la base locale du routeur. Le mot-clé none (optionnel, en dernier recours) autorise l'accès sans authentification si toutes les méthodes précédentes échouent — à utiliser avec prudence en production.

```
! Créer la liste 'AUTH_LOCAL' - méthode : base locale uniquement
Router(config)# aaa authentication login AUTH_LOCAL local

! Pour la console physique - liste séparée recommandée
Router(config)# aaa authentication login CONSOLE_AUTH local none
! ↑ 'none' en fallback : accès garanti même si la base est corrompue
```

Étape 4 — Appliquer les listes aux lignes d'accès

Les listes d'authentification ne s'appliquent pas automatiquement. Il faut les associer explicitement à chaque ligne d'accès : ligne vty (SSH/Telnet) et ligne con 0 (console physique). Cette séparation permet d'avoir des politiques différentes selon le mode d'accès.

```
! Lignes VTY (SSH / Telnet) - 16 sessions simultanées max
Router(config)# line vty 0 15
Router(config-line)# login authentication AUTH_LOCAL
Router(config-line)# transport input ssh
Router(config-line)# exec-timeout 10 0
Router(config-line)# exit

! Console physique
Router(config)# line con 0
Router(config-line)# login authentication CONSOLE_AUTH
Router(config-line)# exec-timeout 5 0
Router(config-line)# exit
```

Étape 5 — Configurer l'autorisation et l'accounting

L'autorisation détermine ce que l'utilisateur peut faire après s'être authentifié. L'accounting enregistre les commandes exécutées. Ces deux fonctions complètent l'authentification pour former le triptyque AAA complet, même dans un contexte purement local.

```
! Autorisation - les droits correspondent au niveau de privilège de l'utilisateur
Router(config)# aaa authorization exec AUTH_LOCAL local
Router(config)# aaa authorization commands 15 AUTH_LOCAL local

! Accounting - journalisation des connexions en local
Router(config)# aaa accounting exec default start-stop local
Router(config)# aaa accounting commands 15 default start-stop local

! Activer SSH (obligatoire pour remplacer Telnet)
Router(config)# ip domain-name dom10.dz
Router(config)# crypto key generate rsa modulus 2048
Router(config)# ip ssh version 2
```

I.3 Vérification et tests

Une fois la configuration appliquée, vérifier que les accès fonctionnent correctement pour chaque niveau de privilège.

```
! Sauvegarder la configuration
Router# write memory

! Vérifier la configuration AAA
Router# show aaa sessions
Router# show users
Router# show running-config | section aaa

! Test SSH depuis une machine externe
user@pc:~$ ssh mshr@192.168.1.1
Password: X123456x
Router# show privilege
Current privilege level is 15

user@pc:~$ ssh g01@192.168.1.1
Password: X123456x
Router> show privilege
Current privilege level is 1
```

I.4 Tableau récapitulatif — AAA local

Fonction AAA	Commande Cisco	Mécanisme	Limitation
Authentication	aaa authentication login AUTH_LOCAL local	Base de données locale du routeur	Dupliquée sur chaque équipement
Authorization	aaa authorization exec AUTH_LOCAL local	Niveau de privilège attribué à l'utilisateur	Pas de politique globale centralisée
Accounting	aaa accounting exec default start-stop local	Logs stockés en mémoire du routeur	Perdus au redémarrage, non centralisés

⚠ Limites de cette approche

- Si le réseau dispose de 50 équipements, les comptes mshr/scher/g01 doivent être créés 50 fois
 - Un changement de mot de passe implique une intervention manuelle sur chaque routeur
 - Les logs d'accounting sont stockés localement et perdus au redémarrage du routeur
 - Aucune cohérence garantie entre équipements : un oubli crée une brèche de sécurité
- C'est précisément pourquoi le modèle basé sur serveur (Partie II) est nécessaire

Partie II — AAA Basé sur Serveur : Configuration du Routeur Cisco

Lorsque l'infrastructure croît, maintenir des comptes locaux sur chaque équipement devient ingérable. La solution est de déléguer les décisions d'authentification à un serveur RADIUS externe. Le routeur ne stocke plus aucun compte utilisateur : il transmet les demandes de connexion au serveur, reçoit la décision, et l'applique. C'est le passage fondamental de l'AAA local à l'AAA centralisé.

Dans ce scénario, le routeur Cisco joue le rôle de NAS (Network Access Server) : il intercepte les tentatives de connexion et les soumet au serveur FreeRADIUS (192.168.1.66). La décision (Access-Accept ou Access-Reject) revient du serveur, avec les attributs d'autorisation (niveau de privilège, VLAN, durée de session). Le routeur n'a plus qu'à appliquer cette décision.

II.1 Déclarer le serveur RADIUS sur le routeur

La première étape est d'indiquer au routeur où se trouve le serveur RADIUS et quel secret partagé utiliser pour sécuriser la communication. Le secret testing123 doit être identique des deux côtés : dans la configuration Cisco et dans le fichier clients.conf de FreeRADIUS.

```
Router(config)# aaa new-model
! ↑ Doit déjà être activé depuis la Partie I

! Déclarer le serveur RADIUS principal
Router(config)# radius server RADIUS_DOM10
Router(config-radius-server)# address ipv4 192.168.1.66 auth-port 1812 acct-port 1813
Router(config-radius-server)# key testing123
Router(config-radius-server)# exit

! Grouper le serveur dans un groupe nommé
Router(config)# aaa group server radius GROUPE_RADIUS
Router(config-sg-radius)# server name RADIUS_DOM10
Router(config-sg-radius)# exit
```

II.2 Définir les méthodes AAA avec fallback local

La méthode group GROUPE_RADIUS interroge le serveur RADIUS en premier. Si le serveur est injoignable (panne réseau, redémarrage), le routeur bascule sur la méthode local comme solution de secours. Ce mécanisme de fallback est essentiel en production : il garantit qu'un administrateur pourra toujours accéder au routeur pour le dépanner, même si le serveur AAA est hors ligne.

```
! Authentification : RADIUS d'abord, base locale en secours
Router(config)# aaa authentication login AUTH_RADIUS group GROUPE_RADIUS local

! Autorisation : les attributs viennent du serveur RADIUS
Router(config)# aaa authorization exec AUTH_RADIUS group GROUPE_RADIUS local
Router(config)# aaa authorization commands 15 default group GROUPE_RADIUS local

! Accounting : le serveur RADIUS enregistre toutes les sessions
Router(config)# aaa accounting exec default start-stop group GROUPE_RADIUS
Router(config)# aaa accounting commands 15 default start-stop group GROUPE_RADIUS
```

II.3 Appliquer aux lignes d'accès et conserver le compte de secours

Le compte de secours admin_local est un compte Cisco IOS local de niveau 15 qui ne passe jamais par RADIUS. Il est réservé à l'administrateur réseau pour les situations d'urgence où le

serveur RADIUS est inaccessible. Sa présence est une bonne pratique de production fortement recommandée.

```
! Compte de secours local - JAMAIS supprimé en production
Router(config)# username admin_local privilege 15 secret X123456x

! Appliquer la liste RADIUS aux VTY (SSH)
Router(config)# line vty 0 15
Router(config-line)# login authentication AUTH_RADIUS
Router(config-line)# authorization exec AUTH_RADIUS
Router(config-line)# transport input ssh
Router(config-line)# exec-timeout 10 0
Router(config-line)# exit

! Console physique - toujours locale (accès d'urgence physique)
Router(config)# line con 0
Router(config-line)# login authentication CONSOLE_AUTH
Router(config-line)# exec-timeout 5 0
Router(config-line)# exit

Router# write memory
```

II.4 Flux complet d'une connexion SSH via RADIUS

Étape	Acteur	Action	Résultat
1	Client SSH	Tentative de connexion SSH sur 192.168.1.1	Invite d'identifiants affichée
2	Routeur (NAS)	Intercepte la demande et envoie un Access-Request RADIUS	Paquet UDP vers 192.168.1.66:1812
3	FreeRADIUS	Reçoit la requête et interroge OpenLDAP (189.168.1.66:389)	Requête LDAP : uid=mshr dans ou=USERS,dc=dom10,dc=dz
4	OpenLDAP	Vérifie l'identité de mshr et son groupe (admins)	Réponse LDAP : OK, groupes=[admins]
5	FreeRADIUS	Applique la politique unlang et renvoie Access-Accept	Attributs : privilege 15, VLAN 10, Session-Timeout=28800
6	Routeur (NAS)	Applique la décision : ouvre la session SSH en mode privilege 15	Connexion établie, shell prompt Router#
7	FreeRADIUS	Enregistre l'événement Accounting-Start	Log horodaté dans /var/log/freeradius/radacct/

II.5 Vérification côté routeur

```
! Tester la connectivité vers le serveur RADIUS
Router# test aaa group GROUPE_RADIUS mshr X123456x new-code
Trying to authenticate with server ...
User successfully authenticated

! Voir les sessions AAA actives
Router# show aaa sessions

! Vérifier la configuration RADIUS
Router# show radius server-group all
```

```
Router# show running-config | section radius
```

```
! Logs d'accounting côté routeur
```

```
Router# show aaa accounting
```

☑ Ce que cette configuration apporte vs. la Partie I

- Un seul point d'administration : mshr/scher/g01 gérés une fois dans LDAP, visibles partout
- Cohérence garantie : même politique sur tous les routeurs déclarant le même serveur RADIUS
- Accounting centralisé : tous les logs dans /var/log/freeradius/ sur le serveur, jamais perdus
- Révocation instantanée : désactiver mshr dans LDAP → accès coupé sur TOUS les équipements
- Fallback préservé : admin_local garantit un accès d'urgence même si le serveur tombe

Partie III — Travail Pratique : Déploiement du Serveur AAA dom10.dz (avec interface graphique)

Les deux parties précédentes ont illustré comment un équipement réseau (le routeur Cisco) consomme un service AAA. Cette troisième partie décrit la construction du service lui-même : le déploiement complet du serveur AAA sur Ubuntu 24.04, comprenant FreeRADIUS, OpenLDAP, SSSD, PAM et Zabbix.

C'est le travail pratique réalisé personnellement dans le cadre de ce stage. Contrairement aux parties précédentes qui concernaient la configuration du NAS (côté routeur), ce TP porte sur la configuration du serveur AAA lui-même — le cerveau de l'infrastructure qui prend toutes les décisions d'accès.

Environnement et architecture du TP

Composant	Technologie	Adresse / Paramètre	Rôle
Serveur AAA	FreeRADIUS 3.x + OpenLDAP	192.168.1.66	Authentification, Autorisation, Accounting
Domaine LDAP	OpenLDAP + phpLDAPadmin	dc=dom10,dc=dz (port 389)	Annuaire centralisé des identités
Machine cliente	Ubuntu 24.04 + SSSD	192.168.1.X (cnode1)	Suppliquant — authentification PAM/SSSD
Supervision	Zabbix Agent 2 + Server 7.4	192.168.1.66 (port 10050)	Monitoring FreeRADIUS et hôtes

Composant	Technologie	Adresse / Paramètre	Rôle
Admin LDAP	cn=admin,dc=dom10,dc=dz	Mot de passe : X123456x	Bind DN pour FreeRADIUS et SSSD
Secret RADIUS	testing123	clients.conf	Authentifie les NAS auprès de FreeRADIUS

Utilisateurs du domaine dom10.dz

Trois utilisateurs ont été créés dans phpLDAPadmin sous ou=USERS,dc=dom10,dc=dz, organisés en groupes distincts pour tester l'autorisation différenciée :

Utilisateur	Groupe LDAP	Rôle simulé	Niveau d'accès RADIUS
mshr	admins	Administrateur réseau	VLAN 10, Session 8h, permit_all, privilege 15
scher	users	Utilisateur standard	VLAN 20, Session 1h, permit_web_only
g01	invites	Invité externe	VLAN 30, Session 30 min, internet_only

TP — Étape 1 : Installation de la pile AAA

Cette première étape installe et configure l'ensemble de la pile logicielle nécessaire sur le serveur 192.168.1.66 : SSSD, PAM, NSS, FreeRADIUS et le module LDAP. Elle prépare le serveur à recevoir les requêtes d'authentification des NAS et à les valider contre l'annuaire OpenLDAP.

```

admin@admin:~$ sudo systemctl status sssd freeradius
[sudo] Mot de passe de admin :
● sssd.service - System Security Services Daemon
   Loaded: loaded (/usr/lib/systemd/system/sss.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-06-05 15:57:28 CET; 58min ago
     Main PID: 1072 (sssd)
       Tasks: 4 (limit: 4543)
      Memory: 56.2M (peak: 57.1M)
         CPU: 1.271s
        CGroup: /system.slice/sss.service
                └─1072 /usr/sbin/sss -i -l logger=files
                  1213 /usr/libexec/sss/sss_be --domain default --uid 0 --gid 0 --logger=files
                  1268 /usr/libexec/sss/sss_nss --uid 0 --gid 0 --logger=files
                  1270 /usr/libexec/sss/sss_pam --uid 0 --gid 0 --logger=files

juin 05 15:57:26 admin.dom10.dz systemd[1]: Starting sssd.service - System Security Services Daemon...
juin 05 15:57:27 admin.dom10.dz sssd[1072]: Starting up
juin 05 15:57:27 admin.dom10.dz sssd_be[1213]: Starting up
juin 05 15:57:28 admin.dom10.dz sssd_nss[1268]: Starting up
juin 05 15:57:28 admin.dom10.dz sssd_pam[1270]: Starting up
juin 05 15:57:28 admin.dom10.dz systemd[1]: Started sssd.service - System Security Services Daemon.
juin 05 15:58:32 admin.dom10.dz sssd_be[1213]: Backend is online

● freeradius.service - FreeRADIUS multi-protocol policy server
   Loaded: loaded (/usr/lib/systemd/system/freeradius.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-06-05 16:50:30 CET; 5min ago
     Docs: man:radiusd(8)
           man:radiusd.conf(5)
           http://wiki.freeradius.org/
           http://networkradius.com/doc/
   Process: 9509 ExecStartPre=/usr/sbin/freeradius_SFREEERADIUS_OPTIONS -Cx -lstdout (code=exited, status=0/SUCCESS)

```

Figure 1 — Services FreeRADIUS et SSSD actifs après installation

1.1 — Installation des paquets

Une seule commande apt installe toute la pile. Elle inclut SSSD (authentification via LDAP), ses outils de diagnostic, les modules d'intégration PAM et NSS, FreeRADIUS et son module LDAP pour la communication avec OpenLDAP.

```
root@serveur:~# sudo apt update
```

```
root@serveur:~# sudo apt install -y \
    sssd sssd-tools sssd-ldap \
    libnss-sss libpam-sss \
    freeradius freeradius-ldap \
    ldap-utils
```

1.2 — Configuration de SSSD

SSSD (System Security Services Daemon) est le pont entre Linux et l'annuaire LDAP. Les paramètres `ldap_default_bind_dn` et `ldap_default_authtok` sont obligatoires sur Ubuntu 24.04 : sans eux, SSSD échoue silencieusement car OpenLDAP refuse les recherches anonymes par défaut. Les droits 600 sont imposés par SSSD lui-même.

```
root@serveur:~# sudo nano /etc/sss/sss.conf
```

```
[sss]
services = nss, pam
config_file_version = 2
domains = default

[domain/default]
id_provider = ldap
auth_provider = ldap
ldap_uri = ldap://192.168.1.66/
ldap_search_base = dc=dom10,dc=dz
cache_credentials = True
enumerate = True
ldap_default_bind_dn = cn=admin,dc=dom10,dc=dz
ldap_default_authtok = X123456x
ldap_id_use_start_tls = False
ldap_auth_disable_tls_never_use_in_production = True
```

```
# Droits obligatoires - SSSD refuse de démarrer sans eux
root@serveur:~# sudo chmod 600 /etc/sss/sss.conf
root@serveur:~# sudo chown root:root /etc/sss/sss.conf
```

1.3 — Configuration de NSS

NSS (Name Service Switch) détermine dans quel ordre le système résout les informations d'identité. En ajoutant `sss` après `files`, on indique à Linux de consulter SSSD (et donc LDAP) après les fichiers locaux. C'est ce qui permet à `getent passwd mshr` de retourner un résultat. Le module `mkhomedir` crée automatiquement le répertoire `/home` à la première connexion.

```
root@serveur:~# sudo nano /etc/nsswitch.conf
```

```
# Modifier ces trois lignes :
passwd:  files sss
group:   files sss
shadow:  files sss
```

```
# Activer mkhomedir (création automatique de /home/mshr à la 1ère connexion)
root@serveur:~# sudo pam-auth-update --enable mkhomedir
root@serveur:~# sudo systemctl restart sssd
```

1.4 — Activation du module LDAP pour FreeRADIUS

FreeRADIUS gère ses modules via des liens symboliques. Le module LDAP est disponible dans `mods-available/` mais inactif par défaut. La création du lien dans `mods-enabled/` l'active sans copier le fichier, ce qui facilite la maintenance.

```
root@serveur:~# sudo ln -s \
    /etc/freeradius/3.0/mods-available/ldap \
    /etc/freeradius/3.0/mods-enabled/ldap

# Vérification
root@serveur:~# ls -la /etc/freeradius/3.0/mods-enabled/ldap
lrwxrwxrwx ... mods-enabled/ldap -> ../mods-available/ldap
```

1.5 — Configuration du module LDAP de FreeRADIUS

Le fichier `mods-available/ldap` indique à FreeRADIUS comment se connecter à OpenLDAP : serveur, port, identifiants de bind, et bases de recherche distinctes pour les utilisateurs (`ou=USERS`) et les groupes (`ou=groups`). La séparation de ces deux bases est importante : elle permet à FreeRADIUS de lire le groupe d'un utilisateur indépendamment de son profil.

```
root@serveur:~# sudo nano /etc/freeradius/3.0/mods-available/ldap

# Dans le bloc ldap { ... } :
server    = '192.168.1.66'
port      = 389
identity  = 'cn=admin,dc=dom10,dc=dz'
password  = 'X123456x'
base_dn   = 'dc=dom10,dc=dz'

# Dans le bloc user { ... } :
base_dn   = "ou=USERS,dc=dom10,dc=dz"
filter    = "(uid=%{%{Stripped-User-Name}:-%{User-Name}})"

# Dans le bloc group { ... } :
base_dn   = "ou=groups,dc=dom10,dc=dz"
filter    = "(objectClass=posixGroup)"
membership_attribute = 'memberUid'
```

1.6 — Déclaration des clients RADIUS

FreeRADIUS n'accepte que les requêtes des clients explicitement déclarés dans `clients.conf`. Le secret `testing123` doit correspondre exactement à celui configuré sur le routeur Cisco (voir Partie II). Un client réseau non déclaré ou avec un secret incorrect recevra une erreur silencieuse.

```
root@serveur:~# sudo nano /etc/freeradius/3.0/clients.conf

# Loopback — pour les tests radtest locaux
client localhost {
    ipaddr = 127.0.0.1
    secret = testing123
}

# Réseau local du laboratoire — tous les NAS de 192.168.1.0/24
client lan-dom10 {
    ipaddr = 192.168.1.0/24
    secret = testing123
}
```

1.7 — Activation de LDAP dans le serveur virtuel default

Dans `sites-enabled/default`, la section `authorize` doit appeler le module `ldap` pour que FreeRADIUS interroge l'annuaire avant de prendre sa décision. Sans cette ligne, FreeRADIUS cherche les utilisateurs uniquement dans ses fichiers locaux et retourne `Access-Reject` pour tous les comptes LDAP.

```
root@serveur:~# sudo nano /etc/freeradius/3.0/sites-enabled/default
```

```
# Dans la section authorize { ... }, ajouter ou décommenter :
authorize {
    ...
    ldap      # <-- interroge OpenLDAP pour chaque demande
    ...
}
```

1.8 — Démarrage et premier test

```
# Ouvrir les ports RADIUS si UFW est actif
root@serveur:~# sudo ufw allow 1812/udp
root@serveur:~# sudo ufw allow 1813/udp
root@serveur:~# sudo ufw reload
```

```
# Démarrer et activer les services
root@serveur:~# sudo systemctl restart sssd freeradius
root@serveur:~# sudo systemctl enable sssd freeradius
root@serveur:~# sudo systemctl status sssd freeradius
```

```
# Premier test d'authentification RADIUS
root@serveur:~# sudo radtest mshr X123456x 127.0.0.1 0 testing123

# Résultat attendu :
Sent Access-Request Id 0 from 0.0.0.0:PORT to 127.0.0.1:1812
Received Access-Accept from host 127.0.0.1 port 1812
```

☑ Validation étape 1

- sssd et freeradius affichent : active (running)
- radtest mshr retourne : Received Access-Accept
- getent passwd mshr retourne : mshr:x:1001:1001:mshr:/home/mshr:/bin/bash
- En cas d'Access-Reject : sudo systemctl stop freeradius && sudo freeradius -X

TP — Étape 2 : Intégration de la machine cliente dans dom10.dz

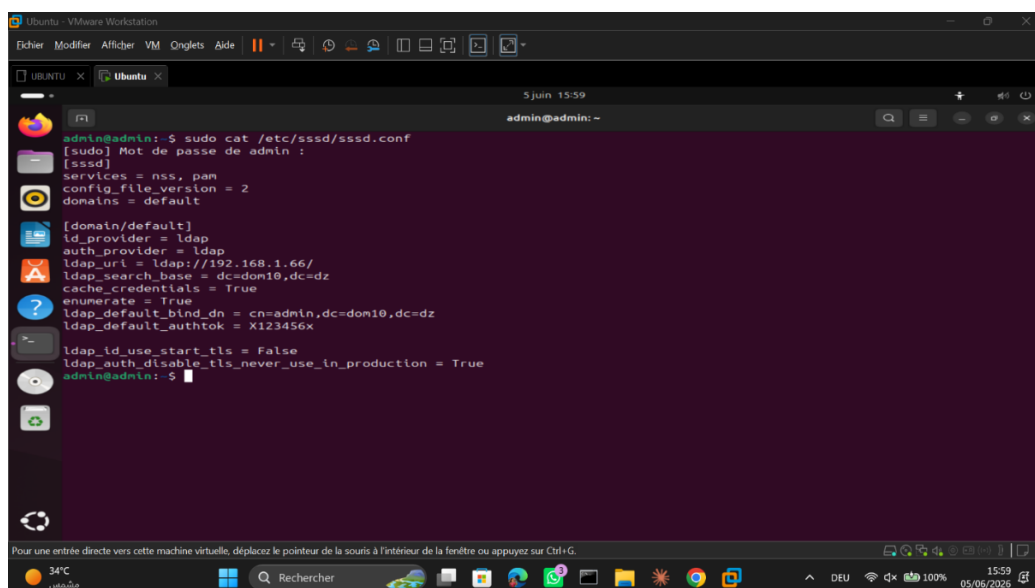


Figure 2 — Configuration de /etc/sss/sss.conf sur le serveur

```

admin@admin:~$ sql: --> INSERT INTO radpostauth (username, pass, reply, authdate) VALUES ('mshr', 'X123456x', 'Access-Accept', '2026-06-05 17:08:31.983950')
admin@admin:~$ sql: Executing query: INSERT INTO radpostauth (username, pass, reply, authdate) VALUES ('mshr', 'X123456x', 'Access-Accept', '2026-06-05 17:08:31.983950')
admin@admin:~$ sql: SQL query returned: success
admin@admin:~$ sql: 1 record(s) updated
admin@admin:~$ rlm_sql (sql): Released connection (1)
admin@admin:~$ [sql] = ok
admin@admin:~$ [exec] = noop
admin@admin:~$ policy remove_reply_message_if_eap {
admin@admin:~$   if (&reply:EAP-Message && &reply:Reply-Message) {
admin@admin:~$     if (&reply:EAP-Message && &reply:Reply-Message) -> FALSE
admin@admin:~$   } else {
admin@admin:~$     [noop] = noop
admin@admin:~$   } # else = noop
admin@admin:~$ } # policy remove_reply_message_if_eap = noop
admin@admin:~$ if (EAP-Key-Name && &reply:EAP-Session-Id) {
admin@admin:~$   if (EAP-Key-Name && &reply:EAP-Session-Id) -> FALSE
admin@admin:~$ } # post-auth = ok
admin@admin:~$ Login OK: [mshr/X123456x] (from client nas1 port 0)
admin@admin:~$ Sent Access-Accept Id 254 from 192.168.1.66:1812 to 192.168.1.66:58051 length 90
admin@admin:~$ Reply-Message := "Admin Connecté - dom10.dz"
admin@admin:~$ Session-Timeout := 28800
admin@admin:~$ Filter-Id := "permit_all"
admin@admin:~$ Tunnel-Private-Group-Id := "10"
admin@admin:~$ Finished request
admin@admin:~$ Waking up in 4.9 seconds.
admin@admin:~$ Cleaning up request packet ID 254 with timestamp +33 due to cleanup_delay was reached
admin@admin:~$ Ready to process requests
  
```

Figure 3a — Sortie FreeRADIUS en mode debug (-X) confirmant l'authentification

```

admin@node1:~$ ssh mshr@192.168.1.66
The authenticity of host '192.168.1.66 (192.168.1.66)' can't be established.
ED25519 key fingerprint is SHA256:LOIKgW4JTHh13v3vc91NpFLXqnd2EbnafKlJ1FBU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.66' (ED25519) to the list of known hosts.
mshr@192.168.1.66's password:
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.17.0-29-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

La maintenance de sécurité étendue pour Applications n'est pas activée.

229 mises à jour peuvent être appliquées immédiatement.
18 de ces mises à jour sont des mises à jour de sécurité.
Pour afficher ces mises à jour supplémentaires, exécuter : apt list --upgradable

21 mises à jour de sécurité supplémentaires peuvent être appliquées avec ESM Apps.
En savoir plus sur l'activation du service ESM Apps at https://ubuntu.com/esm

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/*copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

$ id
uid=1000(melissa) gid=500(admins) groupes=500(admins)
$ whoami
melissa
$ pwd
/home/users/mshr
$
  
```

Figure 3b — Authentification SSH réussie pour l'utilisateur mshr (LDAP)

Après la configuration du serveur, il faut intégrer la machine cliente (cnode1) dans le domaine. Cette opération configure SSSD, PAM et la résolution réseau sur le client, pour qu'il reconnaisse les utilisateurs LDAP et s'authentifie via le serveur 192.168.1.66.

2.1 — Script d'automatisation join_domain.sh

Un script Bash automatise l'ensemble de la procédure d'intégration : installation des paquets, configuration Netplan, configuration SSSD, activation PAM et vérification finale. Utiliser un script garantit la reproductibilité et évite les erreurs manuelles.

```

user@client:~$ sudo nano /tmp/join_domain.sh
user@client:~$ sudo chmod +x /tmp/join_domain.sh
user@client:~$ sudo bash /tmp/join_domain.sh
  
```

```

#!/bin/bash
# Script intégration client LDAP - dom10.dz
set -e
  
```

```

echo "[*] Installation des paquets SSSD..."
sudo apt update
sudo apt install -y sssd sssd-ldap libpam-sss libnss-sss

echo "[*] Configuration réseau Netplan..."
sudo bash -c 'cat > /etc/netplan/01-network-manager-all.yaml' <<EOF
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: true
      dhcp4-overrides:
        use-dns: no
      nameservers:
        addresses: [192.168.1.254, 192.168.1.66, 8.8.8.8]
        search: [dom10.dz]
EOF
sudo chmod 600 /etc/netplan/01-network-manager-all.yaml
sudo netplan apply

echo "[*] Configuration SSSD..."
sudo bash -c 'cat > /etc/sss/sss.conf' <<EOF
[sss]
services = nss, pam
config_file_version = 2
domains = default

[domain/default]
id_provider = ldap
auth_provider = ldap
ldap_uri = ldap://192.168.1.66/
ldap_search_base = dc=dom10,dc=dz
cache_credentials = True
enumerate = True
ldap_default_bind_dn = cn=admin,dc=dom10,dc=dz
ldap_default_authok = X123456x
ldap_id_use_start_tls = False
ldap_auth_disable_tls_never_use_in_production = True
EOF
sudo chown root:root /etc/sss/sss.conf
sudo chmod 600 /etc/sss/sss.conf

echo "[*] Activation PAM mkhomedir et SSSD..."
sudo pam-auth-update --enable mkhomedir
sudo systemctl restart sssd && sudo systemctl enable sssd

# Vérification finale
if getent passwd mshr > /dev/null 2>&1; then
  echo "[+] mshr trouvé via LDAP — intégration réussie."
else
  echo "[!] Échec. Vérifier : sudo journalctl -u sssd -f"
fi

```

2.2 — Vérifications post-intégration

```

# SSSD en ligne et connecté à LDAP
root@client:~# sudo sssctl domain-status default
# Attendu : Online

# Résolution d'un utilisateur LDAP
root@client:~# getent passwd mshr
# Attendu : mshr:x:1001:1001:mshr:/home/mshr:/bin/bash

```

```
# Résolution des groupes
root@client:~# getent group | grep -E "admins|users|invites"
# Attendu : admins:x:1001:mshr / users:x:1002:scher / invites:x:1003:g01

# NSS correctement configuré
root@client:~# cat /etc/nsswitch.conf | grep -E "passwd|group"
# Attendu : passwd: files sss / group: files sss
```

2.3 — Test de connexion SSH avec un compte du domaine

Le test SSH est la validation finale : il sollicite l'ensemble de la chaîne (PAM → SSSD → LDAP).
Le répertoire /home/mshr est créé automatiquement à la première connexion grâce à mkhomedir.

```
# Connexion SSH avec un compte LDAP
user@local:~$ ssh mshr@192.168.1.X
mshr@192.168.1.X's password: X123456x

# Vérifier l'identité après connexion
mshr@client:~$ id
uid=1001(mshr) gid=1001(mshr) groups=1001(mshr)

mshr@client:~$ whoami
mshr

# Logs d'authentification PAM
root@client:~# sudo tail -f /var/log/auth.log | grep -E "sssd|pam|sshd"
```

☒ Validation étape 2

- sssctl domain-status default affiche : Online
- getent passwd mshr retourne le profil LDAP complet
- La connexion SSH réussit avec les credentials LDAP
- id affiche l'uid/gid LDAP (non local) : uid=1001(mshr)
- /home/mshr est créé automatiquement à la première connexion

TP — Étape 3 : Authentification RADIUS — Tests et Validation

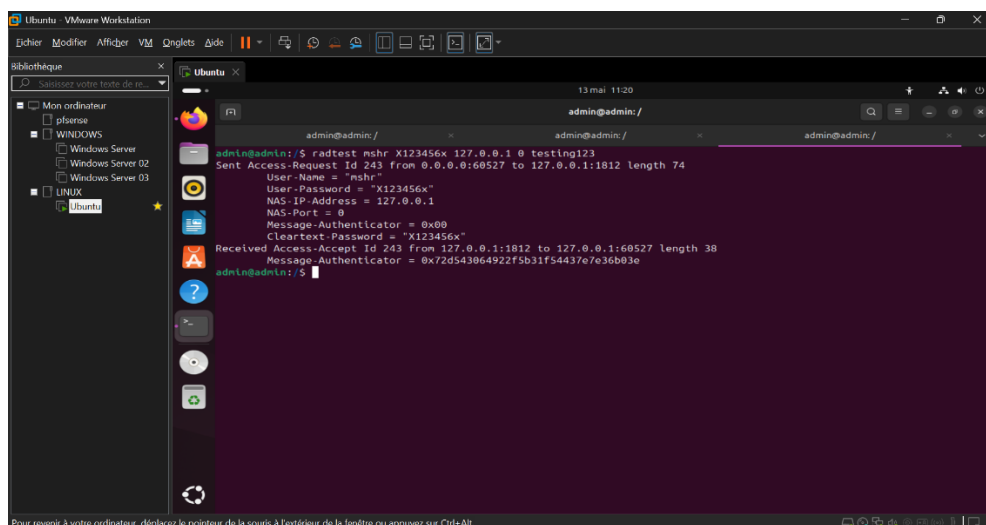


Figure 4 — Test radtest : Access-Accept pour mshr

Cette étape valide que FreeRADIUS authentifie correctement les trois utilisateurs du domaine via OpenLDAP. On distingue deux flux : l'authentification utilisateur via RADIUS (testée avec radtest) et l'authentification machine via SSSD/PAM (testée avec getent et SSH).

3.1 — Vérification de la configuration LDAP

```
# Confirmer les paramètres LDAP actifs dans FreeRADIUS
root@serveur:~# sudo grep -E "server|identity|password|base_dn" \
/etc/freeradius/3.0/mods-available/ldap

# Résultat attendu :
server      = '192.168.1.66'
identity    = 'cn=admin,dc=dom10,dc=dz'
password    = 'X123456x'
base_dn     = 'dc=dom10,dc=dz'
```

3.2 — Tests radtest des trois utilisateurs

```
# Test mshr - groupe admins
root@serveur:~# sudo radtest mshr X123456x 192.168.1.66 0 testing123

# Test scher - groupe users
root@serveur:~# sudo radtest scher X123456x 192.168.1.66 0 testing123

# Test g01 - groupe invites
root@serveur:~# sudo radtest g01 X123456x 192.168.1.66 0 testing123

# Résultat attendu pour chaque test :
Received Access-Accept from host 192.168.1.66 port 1812

# En cas d'Access-Reject - démarrer le mode debug pour diagnostiquer :
root@serveur:~# sudo systemctl stop freeradius
root@serveur:~# sudo freeradius -X
# Dans un autre terminal, relancer le radtest
```

3.3 — Vérification SSSD côté client

```
# Résolution utilisateur LDAP
root@client:~# getent passwd mshr
mshr:x:1001:1001:mshr:/home/mshr:/bin/bash

# Connexion directe avec su
root@client:~# sudo su - mshr
# → Saisir : X123456x
mshr@client:~$ id
uid=1001(mshr) gid=1001(mshr) groups=1001(mshr)
```

Le flux d'authentification complet pour une connexion SSH peut se résumer ainsi : la machine cliente (PAM) soumet les credentials à SSSD, qui contacte OpenLDAP (189.168.1.66:389). Pour une authentification RADIUS (NAS → FreeRADIUS), le NAS envoie un Access-Request au port 1812, FreeRADIUS interroge LDAP, et retourne Access-Accept avec les attributs d'autorisation.

TP — Étape 4 : Autorisation par Groupe LDAP

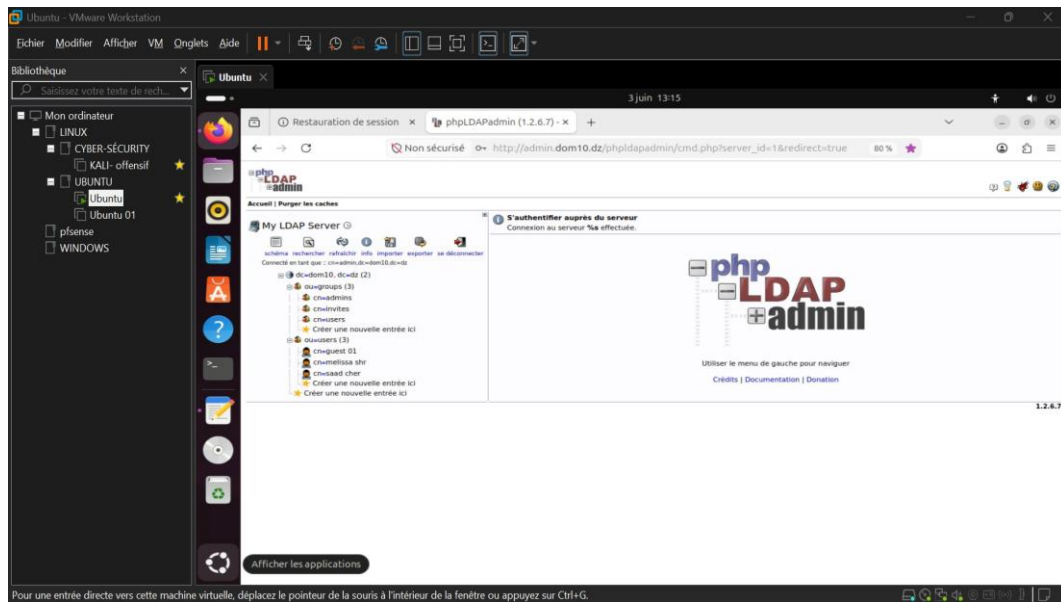


Figure 7 — Arborescence LDAP avec les groupes admins, users et invites

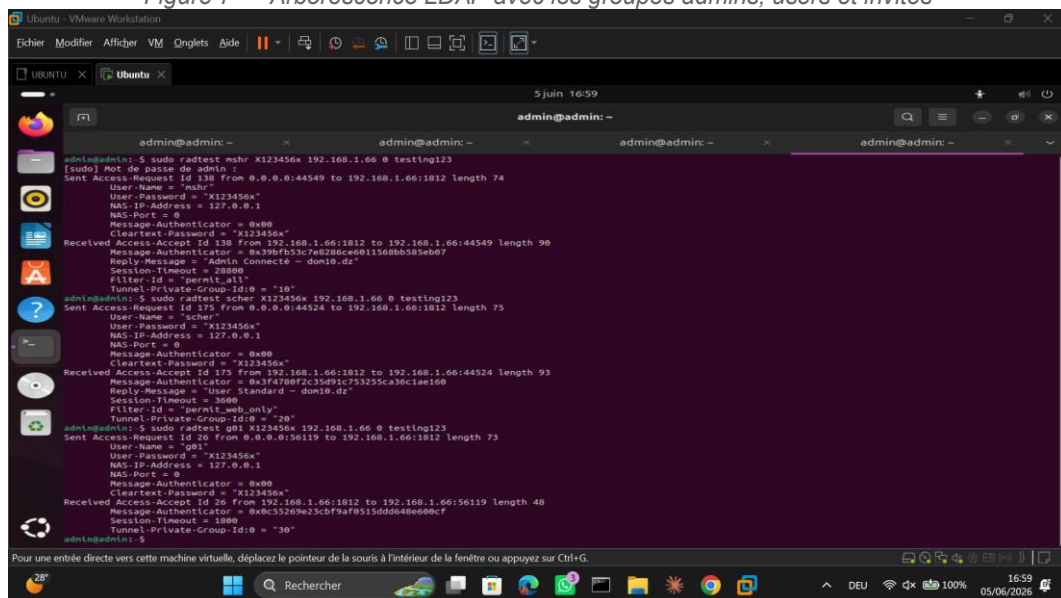


Figure 8 — Tests radtest : autorisation par groupe (mshr → VLAN 10, scher → VLAN 20, g01 → VLAN 30)

L'autorisation est la deuxième fonction du modèle AAA. Elle définit ce que chaque utilisateur est autorisé à faire, en fonction de son appartenance à un groupe LDAP. FreeRADIUS propose deux méthodes : le fichier users (statique) et unlang (dynamique). La méthode unlang est utilisée dans ce TP car elle lit les groupes LDAP en temps réel et permet des conditions complexes.

⚠ Règle absolue

Ne jamais activer les deux méthodes (fichier users ET unlang) simultanément.

Cela provoque des doublons d'attributs, des règles écrasées et un comportement imprévisible.

Ce TP utilise exclusivement unlang (méthode 2).

4.1 — Catalogue des attributs d'autorisation RADIUS

Attribut	Rôle	Exemple
Session-Timeout	Durée maximale de session (secondes)	28800 = 8h / 3600 = 1h / 1800 = 30 min
Idle-Timeout	Déconnexion si inactif (secondes)	3600 = 1h / 600 = 10 min / 300 = 5 min
Filter-Id	Identifiant de filtre (politique firewall/ACL)	permit_all / permit_web_only / internet_only
Tunnel-Type	Type de tunnel pour l'assignation VLAN	VLAN
Tunnel-Medium-Type	Medium physique du tunnel	IEEE-802
Tunnel-Private-Group-Id	Numéro du VLAN attribué à l'utilisateur	10 (admins) / 20 (users) / 30 (invites)
Reply-Message	Message affiché après connexion réussie	Bienvenue Administrateur — dom10.dz
Port-Limit	Nombre max de sessions simultanées	5 (admins) / 2 (users) / 1 (invites)

4.2 — Politique d'autorisation définie

Groupe	Utilisateur	VLAN	Session max	Filtre	Sessions max
admins	mshr	VLAN 10	8 heures (28800s)	permit_all	5
users	scher	VLAN 20	1 heure (3600s)	permit_web_only	2
invites	g01	VLAN 30	30 minutes (1800s)	internet_only	1

4.3 — Configuration unlang dans sites-enabled/default

Ajouter le bloc suivant dans la section post-auth { ... } du fichier /etc/freeradius/3.0/sites-enabled/default. Les conditions sont évaluées séquentiellement, et la clause else finale rejette explicitement tout utilisateur n'appartenant à aucun groupe autorisé.

```
root@serveur:~# sudo nano /etc/freeradius/3.0/sites-enabled/default

# Dans post-auth { ... } – ajouter ce bloc complet :

# — ADMINS : mshr – accès total, session 8h —————
if (LDAP-Group == "admins") {
    update reply {
        Reply-Message           := "Bienvenue Administrateur – dom10.dz"
        Session-Timeout         := 28800
        Idle-Timeout             := 3600
        Filter-Id                := "permit_all"
        Port-Limit               := 5
        Tunnel-Type              := VLAN
        Tunnel-Medium-Type       := IEEE-802
        Tunnel-Private-Group-Id := "10"
    }
}
# — USERS : scher – web uniquement, session 1h —————
elseif (LDAP-Group == "users") {
    update reply {
        Reply-Message           := "Bienvenue Utilisateur – Acces Standard"
        Session-Timeout         := 3600
        Idle-Timeout             := 600
        Filter-Id                := "permit_web_only"
    }
}
```

```
        Port-Limit           := 2
        Tunnel-Type          := VLAN
        Tunnel-Medium-Type   := IEEE-802
        Tunnel-Private-Group-Id := "20"
    }
}
# — INVITES : g01 - minimal, 30 min —————
elsif (LDAP-Group == "invites") {
    update reply {
        Reply-Message           := "Acces Invite - Limite 30 min"
        Session-Timeout         := 1800
        Idle-Timeout            := 300
        Filter-Id               := "internet_only"
        Port-Limit              := 1
        Tunnel-Type             := VLAN
        Tunnel-Medium-Type      := IEEE-802
        Tunnel-Private-Group-Id := "30"
    }
}
# — REJET EXPLICITE - tout utilisateur hors groupe —————
else {
    update reply {
        Reply-Message := "Acces refuse - Groupe non autorise"
    }
    reject
}
```

```
root@serveur:~# sudo systemctl restart freeradius
```

4.4 — Validation des politiques

```
# mshr (admins) → Session-Timeout=28800, VLAN=10, Filter=permit_all
root@serveur:~# sudo radtest mshr X123456x 192.168.1.66 0 testing123

# scher (users) → Session-Timeout=3600, VLAN=20, Filter=permit_web_only
root@serveur:~# sudo radtest scher X123456x 192.168.1.66 0 testing123

# g01 (invites) → Session-Timeout=1800, VLAN=30, Filter=internet_only
root@serveur:~# sudo radtest g01 X123456x 192.168.1.66 0 testing123

# Test de rejet - utilisateur hors groupe
root@serveur:~# sudo radtest inconnu wrongpass 192.168.1.66 0 testing123
# Attendu : Access-Reject | Reply-Message="Acces refuse - Groupe non autorise"
```

TP — Étape 5 : Accounting — Journalisation des Sessions

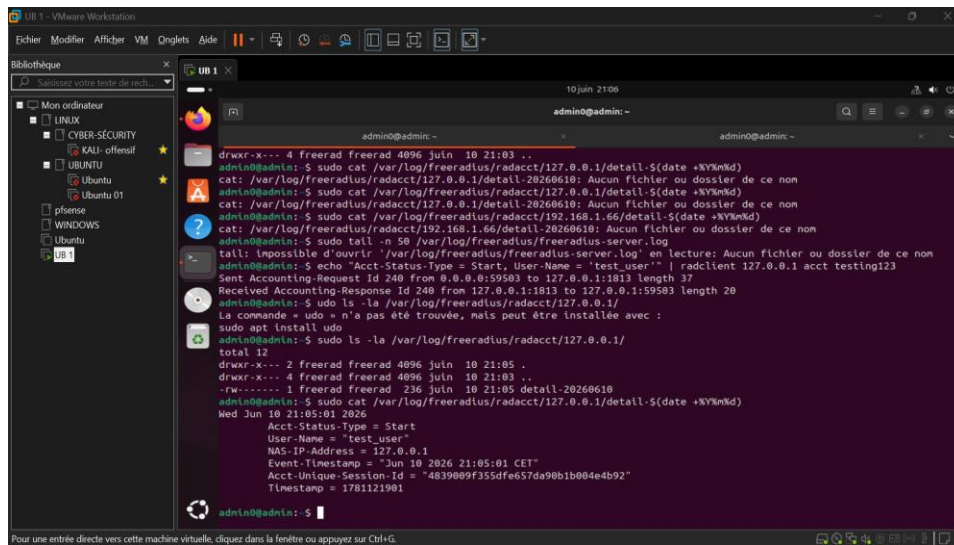


Figure 9 — Fichier detail et table radacct : journalisation des sessions RADIUS

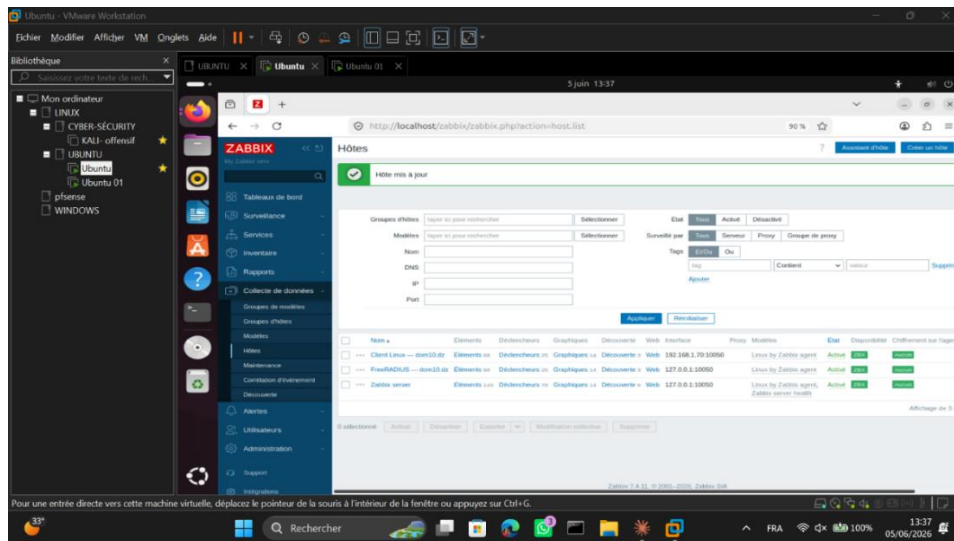


Figure 11 — Interface Zabbix : hôtes FreeRADIUS et client en supervision active

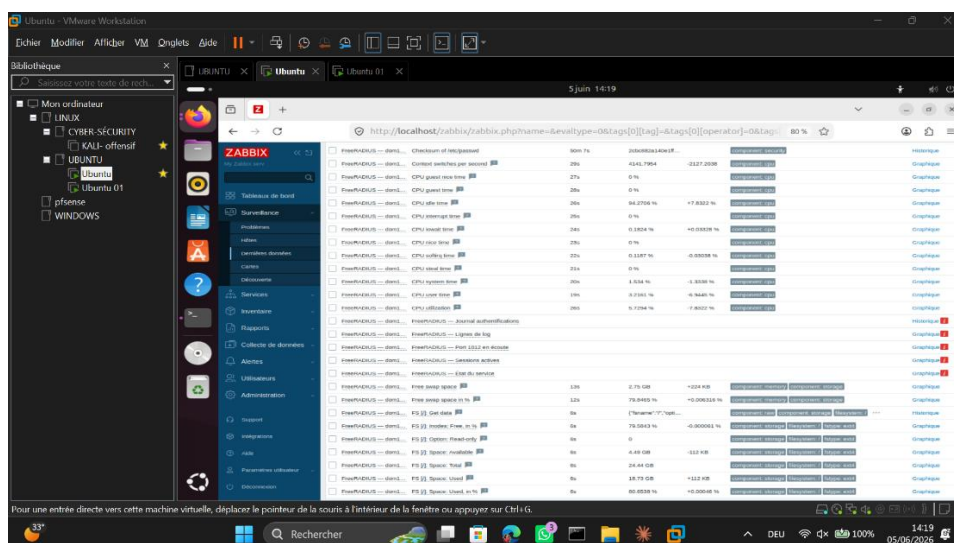


Figure 12 — Déclencheurs Zabbix configurés pour les alertes FreeRADIUS

L'accounting est la troisième et dernière fonction du modèle AAA. Elle enregistre automatiquement chaque événement de session RADIUS : qui s'est connecté, depuis quelle adresse, à quelle heure et pour quelle durée. Cette étape est divisée en deux parties : la journalisation fichier (module detail) et la supervision via Zabbix.

Partie 1 — Journalisation dans les fichiers (module detail)

5.1 — Préparation de l'arborescence de stockage

FreeRADIUS crée les fichiers de log automatiquement, mais l'arborescence doit exister et le service freerad doit avoir les droits d'écriture. Sans ces prérequis, le module detail échoue silencieusement.

```
root@serveur:~# sudo mkdir -p /var/log/freeradius/radacct/192.168.1.66
root@serveur:~# sudo chown -R freerad:freerad /var/log/freeradius/radacct
root@serveur:~# sudo chmod -R 750 /var/log/freeradius/radacct
root@serveur:~# ls -la /var/log/freeradius/radacct/
```

5.2 — Activation du module detail

```
# Vérifier si le lien existe déjà
root@serveur:~# ls /etc/freeradius/3.0/mods-enabled/detail

# Si absent, créer le lien symbolique
root@serveur:~# sudo ln -s \
    /etc/freeradius/3.0/mods-available/detail \
    /etc/freeradius/3.0/mods-enabled/detail

# Vérifier la configuration (chemin des fichiers de log)
root@serveur:~# sudo nano /etc/freeradius/3.0/mods-available/detail

# Ligne critique à vérifier :
detail {
    filename = ${radacctdir}/${Client-IP-Address}/detail-%Y%m%d
    permissions = 0600
}
# ${radacctdir} = /var/log/freeradius/radacct
```

5.3 — Activation dans la section accounting

```
root@serveur:~# sudo nano /etc/freeradius/3.0/sites-enabled/default

# Dans accounting { ... } — décommenter ces modules :
accounting {
    detail      # logs dans /var/log/freeradius/radacct/192.168.1.66/detail-YYYYMMDD
    unix        # logs dans /var/log/wtmp
    radutmp     # suivi des sessions actives
}

root@serveur:~# sudo systemctl restart freeradius
root@serveur:~# sudo systemctl status freeradius
```

5.4 — Script de correction automatique accounting_fix.sh

Ce script applique toutes les corrections accounting en une seule commande : création des répertoires, activation du module detail, décommentage intelligent (sans créer de doublons), test de configuration et redémarrage. Il fait une sauvegarde et restaure automatiquement en cas d'erreur.

```

root@serveur:~# sudo nano /tmp/accounting_fix.sh
root@serveur:~# sudo chmod +x /tmp/accounting_fix.sh
root@serveur:~# sudo bash /tmp/accounting_fix.sh

#!/bin/bash
# Script correction accounting FreeRADIUS - sécurisé (sans doublons)
set -e
echo "=== FreeRADIUS ACCOUNTING FIX ==="

# 1. Arrêt service
systemctl stop freeradius || true
pkill freeradius || true

# 2. Arborescence logs
mkdir -p /var/log/freeradius/radacct/192.168.1.66
chown -R freerad:freerad /var/log/freeradius
chmod -R 755 /var/log/freeradius

# 3. Activation module detail
[ ! -e /etc/freeradius/3.0/mods-enabled/detail ] && \
    ln -s /etc/freeradius/3.0/mods-available/detail \
        /etc/freeradius/3.0/mods-enabled/detail

# 4. Test de configuration
if freeradius -XC; then
    echo "Configuration valide."
else
    echo "ERREUR - restauration sauvegarde."
    exit 1
fi

# 5. Redémarrage
systemctl restart freeradius
systemctl enable freeradius
systemctl status freeradius --no-pager
echo "=== ACCOUNTING FIX APPLIQUÉ AVEC SUCCÈS ==="

```

5.5 — Test en temps réel avec tail

Ouvrir deux terminaux simultanément : le premier suit le fichier detail en temps réel, le second génère une authentification RADIUS. Les champs User-Name et Acct-Status-Type doivent apparaître dans le terminal 1.

```

# Terminal 1 - suivre les logs en temps réel
root@serveur:~# sudo tail -f \
    /var/log/freeradius/radacct/192.168.1.66/detail-$(date +%Y%m%d)

# Terminal 2 - générer une authentification
root@serveur:~# sudo radtest mshr X123456x 192.168.1.66 0 testing123

```

5.6 — Script radwho personnalisé — sessions actives

L'outil système radwho retourne souvent vide sur les installations modernes. Ce script personnalisé parse les fichiers detail pour afficher les utilisateurs réellement connectés (sessions Start sans Stop correspondant).

```

root@serveur:~# sudo nano /usr/local/bin/radwho

#!/bin/bash
LOGFILE="/var/log/freeradius/radacct/192.168.1.66/detail-$(date +%Y%m%d)"

[ ! -r "$LOGFILE" ] && echo "Erreur : fichier inaccessible" && exit 1

```

```
rm -f /tmp/user_*

grep -E "User-Name|Acct-Status-Type" "$LOGFILE" \
  | sed 's/"//g' | awk '{print $3}' > /tmp/parsed_log

while read user && read status; do
  [ "$status" = "Start" ] && touch "/tmp/user_${user}" \
    || rm -f "/tmp/user_${user}"
done < /tmp/parsed_log

echo "Utilisateurs actuellement connectés :"
ls /tmp/user_* 2>/dev/null | xargs -I{} basename {} | sed 's/user_/'
echo "Total : $(ls /tmp/user_* 2>/dev/null | wc -l) session(s) active(s)"

root@serveur:~# sudo chmod +x /usr/local/bin/radwho
root@serveur:~# sudo radwho

# Résultat attendu :
Utilisateurs actuellement connectés :
mshr
Total : 1 session(s) active(s)
```

Partie 2 — Supervision via Zabbix

Zabbix est déjà installé sur 192.168.1.66 (MySQL, dépôt officiel Zabbix 7.4). Cette partie ajoute la supervision de FreeRADIUS dans le dashboard Zabbix via des métriques personnalisées (UserParameters) et crée les déclencheurs d'alerte.

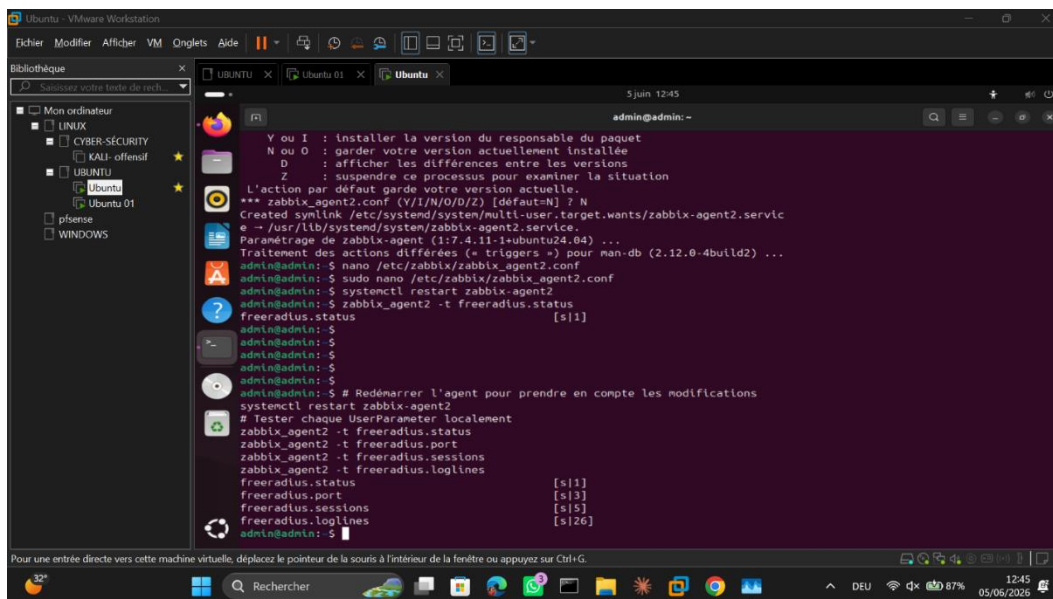


Figure 10 — Configuration et vérification de l'agent Zabbix pour la supervision de FreeRADIUS

5.7 — Installation de Zabbix Agent 2 sur le client

```
# Sur la machine client Linux (192.168.1.X)
root@client:~# wget https://repo.zabbix.com/zabbix/7.4/release/ubuntu/pool/\
main/z/zabbix-release/zabbix-release_latest_7.4+ubuntu24.04_all.deb
root@client:~# dpkg -i zabbix-release_latest_7.4+ubuntu24.04_all.deb
root@client:~# apt update && apt install zabbix-agent2 -y

# Configuration de l'agent
root@client:~# nano /etc/zabbix/zabbix_agent2.conf
Server = 192.168.1.66
ServerActive = 192.168.1.66
```

```

Hostname      = client-linux
ListenPort    = 10050

root@client:~# systemctl enable --now zabbix-agent2
root@client:~# ufw allow 10050/tcp # si UFW actif

```

5.8 — UserParameters FreeRADIUS sur le serveur

```

root@serveur:~# nano /etc/zabbix/zabbix_agent2.conf

# Paramètres de base
Server      = 127.0.0.1
ServerActive = 127.0.0.1
Hostname    = srv-freeradius
ListenPort  = 10050

# UserParameters FreeRADIUS
UserParameter=freeradius.status,systemctl is-active freeradius | grep -c '^active'
UserParameter=freeradius.port,ss -uln | grep -c 1812
UserParameter=freeradius.sessions,radwho 2>/dev/null | tail -n +1 | grep -c '.' ||
echo 0
UserParameter=freeradius.loglines,wc -l < /var/log/freeradius/radius.log
UserParameter=freeradius.last_result,tail -20 /var/log/freeradius/radius.log
2>/dev/null | grep -c 'Access-Accept' || echo 0

root@serveur:~# systemctl restart zabbix-agent2

# Tests locaux des métriques
root@serveur:~# zabbix_agent2 -t freeradius.status      # Attendu : 1
root@serveur:~# zabbix_agent2 -t freeradius.port        # Attendu : 1
root@serveur:~# zabbix_agent2 -t freeradius.sessions    # Attendu : 0 (hors session)
root@serveur:~# zabbix_agent2 -t freeradius.loglines    # Attendu : nombre de lignes

```

5.9 — Ajout de l'hôte serveur FreeRADIUS dans Zabbix

Se connecter à l'interface web Zabbix : <http://192.168.1.66/zabbix> (login : Admin / zabbix). On va d'abord déclarer le serveur FreeRADIUS comme hôte surveillé. Puisque FreeRADIUS et Zabbix tournent sur la même machine, l'interface agent utilise l'adresse de loopback 127.0.0.1.

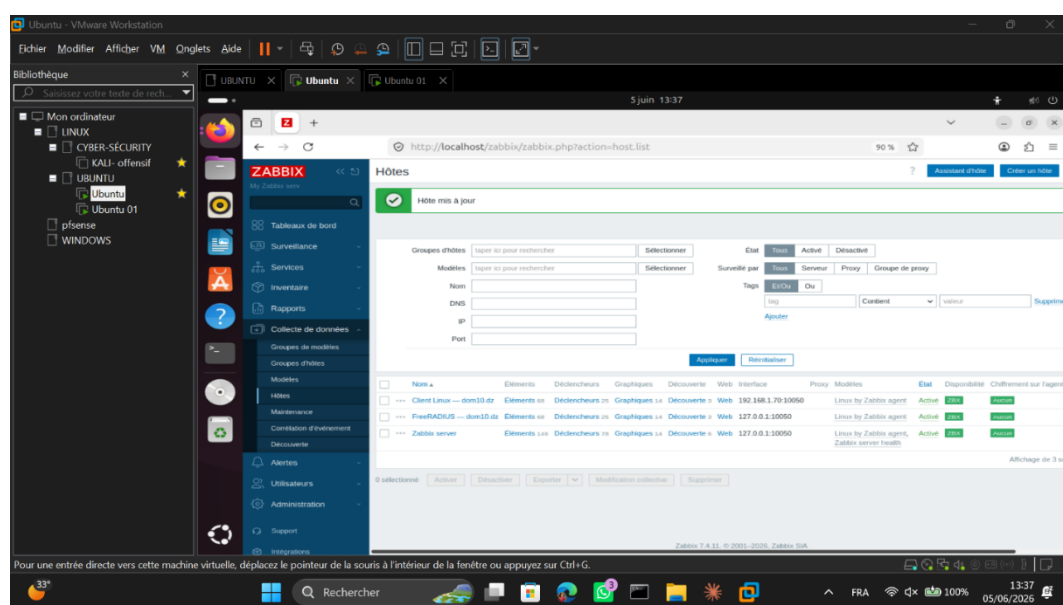


Figure 11b — Ajout de l'hôte serveur FreeRADIUS dans l'interface web Zabbix

Collecte de données → Hôtes → Créer un hôte

ONGLET « HÔTE »

Nom de l'hôte	srv-freeradius
Nom visible	FreeRADIUS — dom10.dz
Groupe	Sélectionner → Linux servers (ou créer : Serveurs RADIUS)
Interface Agent	Ajouter → Type : Agent · IP : 127.0.0.1 · Port : 10050

ONGLET « MODÈLES »

Lier les modèles	Sélectionner → Linux by Zabbix agent → cocher → Sélectionner
Enregistrer	Cliquer Ajouter en bas de la page

⚠ Le champ Nom de l'hôte (srv-freeradius) doit correspondre exactement au paramètre Hostname=srv-freeradius dans /etc/zabbix/zabbix_agent2.conf. Une différence de casse ou d'espace et l'hôte restera rouge dans Zabbix.

☒ Après quelques secondes, l'hôte apparaît en vert dans la liste. Si rouge : vérifier que zabbix-agent2 tourne (systemctl status zabbix-agent2) et qu'aucun firewall ne bloque le port 10050.

5.9b — Ajout de l'hôte client Linux dans Zabbix

On répète la même opération pour la machine client (192.168.1.X). Cet hôte permet de vérifier la connectivité réseau du client et de superviser ses ressources (CPU, mémoire, disque).

Collecte de données → Hôtes → Créer un hôte

ONGLET « HÔTE »

Nom de l'hôte	client-linux (doit correspondre à Hostname= dans zabbix_agent2.conf du client)
Nom visible	Client Linux — dom10.dz
Groupe	Sélectionner → Linux servers
Interface Agent	Ajouter → Type : Agent · IP : 192.168.1.X · Port : 10050

ONGLET « MODÈLES »

Lier les modèles	Sélectionner → Linux by Zabbix agent → cocher → Sélectionner
Enregistrer	Cliquer Ajouter en bas de la page

☒ Les deux hôtes (srv-freeradius et client-linux) apparaissent en vert dans la liste des hôtes Zabbix.

5.10 — Créer les éléments (items) de monitoring FreeRADIUS

Pour chaque UserParameter défini à l'étape 5.8, on crée un élément dans Zabbix. Chaque élément correspond à une métrique FreeRADIUS collectée périodiquement par le serveur Zabbix.

Navigation : Collecte de données → Hôtes → srv-freeradius → Éléments → Créer un élément

Nom	Type	Clé	Type d'info	Intervalle
État du service FreeRADIUS	Agent Zabbix	freeradius.status	Numérique	30s
Port 1812 UDP en écoute	Agent Zabbix	freeradius.port	Numérique	60s
Sessions RADIUS actives	Agent Zabbix	freeradius.sessions	Numérique	30s
Lignes de log (activité globale)	Agent Zabbix	freeradius.loglines	Numérique	60s
Journal authentications	Agent Zabbix (actif)	log[/var/log/freeradius/radius.log]	Journal (Log)	5s

☑ Une fois les 5 éléments créés : Collecte de données → Hôtes → srv-freeradius → Dernières données. Après le prochain cycle (max 60s), les valeurs apparaissent : freeradius.status=1, freeradius.port=1. Le journal affiche les Access-Accept et Access-Reject en temps réel.

Navigation : Collecte de données → Hôtes → srv-freeradius → Déclencheurs → Créer un déclencheur

Nom	Sévérité	Expression	Description
FreeRADIUS est arrêté !	Élevée (High)	last(/srv-freeradius/freeradius.status)=0	0 = service systemd inactif
Port RADIUS 1812 inaccessible !	Désastre (Disaster)	last(/srv-freeradius/freeradius.port)=0	Port 1812 UDP fermé = aucune auth possible
Sessions RADIUS élevées (>50)	Avertissement	last(/srv-freeradius/freeradius.sessions)>50	Charge inhabituellement élevée (optionnel)

☑ Une fois les triggers créés : Surveillance → Problèmes. Tester en arrêtant FreeRADIUS (systemctl stop freeradius) : une alerte rouge « FreeRADIUS est arrêté ! » et « Port RADIUS 1812 inaccessible ! » apparaissent instantanément dans le dashboard Zabbix.

5.12 — Vérification finale — FreeRADIUS visible dans Zabbix

Générer du trafic RADIUS depuis le serveur pour vérifier que Zabbix collecte correctement les métriques et que le journal d'authentification se remplit en temps réel.

Côté serveur — générer du trafic RADIUS pour tester :

```
# 1. Tester une authentification RADIUS (génère une entrée dans le log)
radtest mshr motdepasse 192.168.1.66 0 testing123
```

```
# 2. Vérifier que le log est bien écrit
tail -5 /var/log/freeradius/radius.log
```

```
# 3. Vérifier que Zabbix Agent collecte bien la métrique
zabbix_agent2 -t freeradius.last_result
```

Ce que tu dois voir dans Surveillance → Dernières données → srv-freeradius :

freeradius.status 1 — service actif

freeradius.port 1 — port 1812 UDP en écoute

freeradius.sessions nombre de sessions actives (souvent 0 hors connexion réelle)

freeradius.loglines nombre de lignes dans radius.log (augmente à chaque authentification)

Journal auth Access-Accept et Access-Reject apparaissent avec timestamp en temps réel

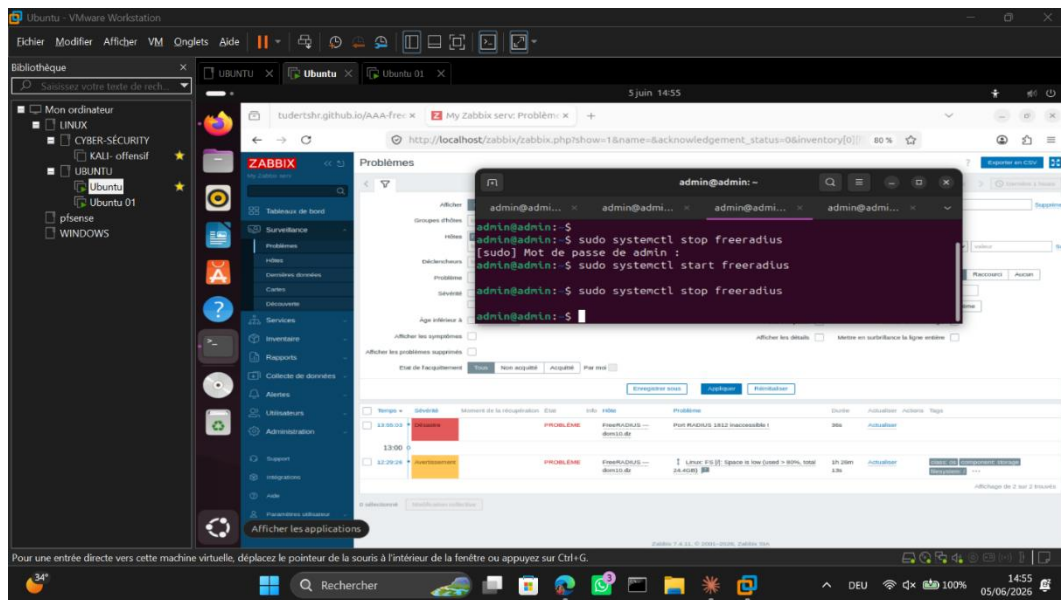


Figure 13a — Détection de l'arrêt du service FreeRADIUS (port 1812 inaccessible) par Zabbix

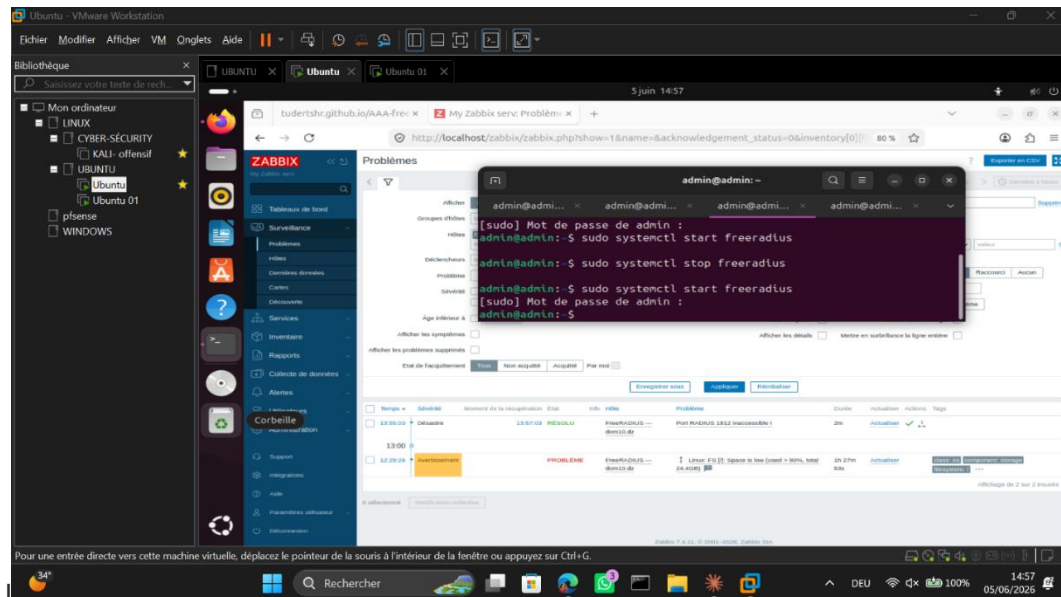


Figure 13b — Rétablissement du service FreeRADIUS et résolution de l'alerte dans Zabbix

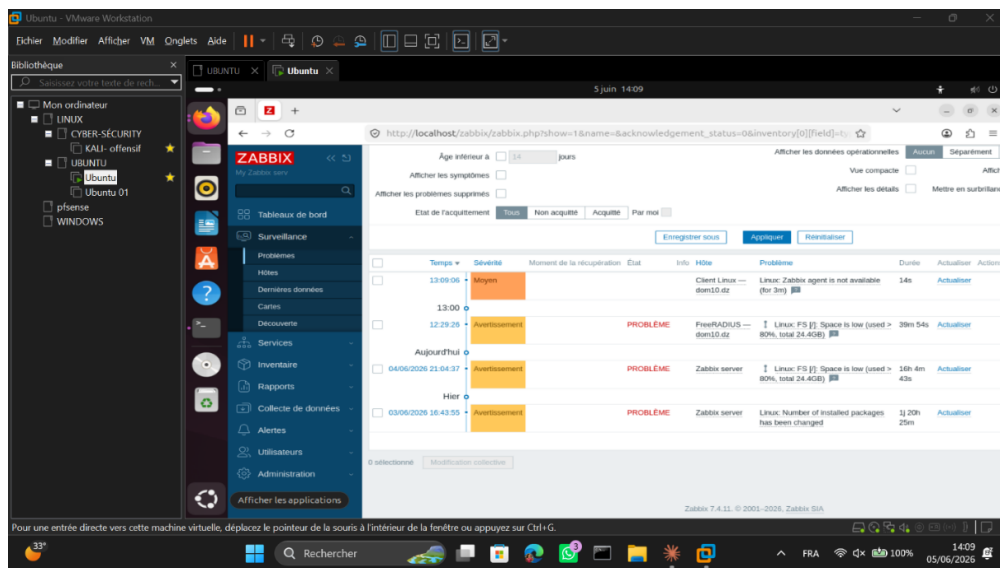


Figure 14a — Détection de l'arrêt de la machine cliente par Zabbix

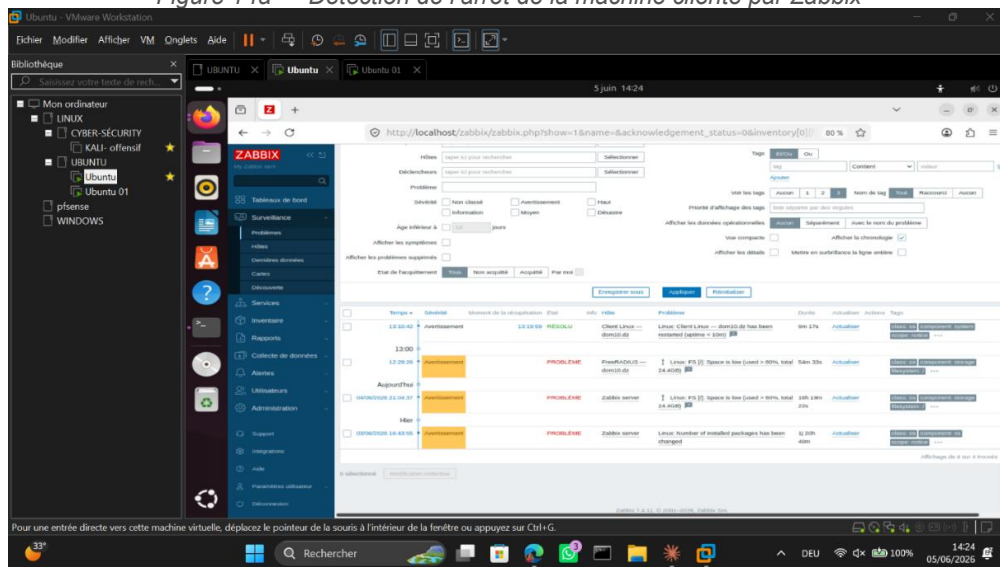


Figure 14b — Remise en service de la machine cliente et résolution de l'alerte dans Zabbix

TP — Étape 6 : Authentification Wi-Fi 802.1X avec EAP-TTLS

Cette étape étend le contrôle AAA au réseau sans fil. EAP-TTLS (Tunneled TLS) crée un tunnel TLS chiffré entre le client et FreeRADIUS avant de transmettre les credentials en clair à l'intérieur du tunnel. C'est la méthode la plus compatible avec OpenLDAP : contrairement à EAP-PEAP/MSCHAPv2, EAP-TTLS/PAP fonctionne nativement avec les mots de passe LDAP sans hash NTLM.

6.1 — Génération des certificats TLS auto-signés

FreeRADIUS inclut un système de génération de certificats complet dans `/etc/freeradius/3.0/certs/`. La commande `make all` génère l'autorité de certification (`ca.pem`), le certificat serveur (`server.pem`), les paramètres Diffie-Hellman (`dh`) et un certificat client optionnel (`client.pem`).

```
root@serveur:~# cd /etc/freeradius/3.0/certs/
root@serveur:/etc/freeradius/3.0/certs/# sudo make all
```

```
# Vérifier les fichiers générés
root@serveur:/etc/freeradius/3.0/certs/# ls -la *.pem dh

# Résultat attendu :
ca.pem      ← Autorité de Certification (à distribuer aux clients)
server.pem  ← Certificat + clé privée du serveur RADIUS
client.pem  ← Certificat client (pour EAP-TLS si besoin)
dh          ← Paramètres Diffie-Hellman
```

6.2 — Configuration du module EAP-TTLS

Modifier `mods-available/eap` pour définir EAP-TTLS comme méthode par défaut. La phase interne utilise PAP (et non MSCHAPv2) car OpenLDAP stocke les mots de passe en clair (ou hash LDAP), incompatibles avec MSCHAPv2 qui requiert un hash NTLM.

```
root@serveur:~# sudo nano /etc/freeradius/3.0/mods-available/eap

# Paramètres à modifier :
default_eap_type = ttls

tls-config tls-common {
    private_key_file = ${certdir}/server.pem
    certificate_file = ${certdir}/server.pem
    ca_file           = ${cadir}/ca.pem
    dh_file           = ${certdir}/dh
}

ttls {
    tls = tls-common
    default_eap_type = pap
    # PAP obligatoire avec OpenLDAP
    virtual_server = "inner-tunnel"
}

# Vérifier que inner-tunnel est activé (traite les credentials dans le tunnel)
root@serveur:~# ls -la /etc/freeradius/3.0/sites-enabled/inner-tunnel

# Si absent :
root@serveur:~# sudo ln -s \
    /etc/freeradius/3.0/sites-available/inner-tunnel \
    /etc/freeradius/3.0/sites-enabled/inner-tunnel

# Tester la configuration puis redémarrer
root@serveur:~# sudo freeradius -XC
# Attendu : Configuration appears to be OK.

root@serveur:~# sudo systemctl restart freeradius
```

6.3 — Compilation de eapol_test sur la machine cliente

`eapol_test` n'est pas disponible dans les dépôts Ubuntu. Il fait partie du projet `wpa_supplicant` et doit être compilé depuis les sources. Cette opération se fait sur la machine cliente (`cnodel1`).

```
# Installer les dépendances de compilation
root@cnodel1:~# sudo apt install -y \
    build-essential libssl-dev \
    libnl-genl-3-dev libnl-route-3-dev wget

# Télécharger et décompresser les sources
root@cnodel1:~# wget https://w1.fi/releases/wpa_supplicant-2.10.tar.gz
root@cnodel1:~# tar -xzf wpa_supplicant-2.10.tar.gz
root@cnodel1:~# cd wpa_supplicant-2.10/wpa_supplicant
```

```
# Configurer et compiler eapol_test
root@cnode1:~# cp defconfig .config
root@cnode1:~# echo 'CONFIG_EAPOL_TEST=y' >> .config
root@cnode1:~# echo 'CFLAGS += -Wno-error=deprecated-declarations' >> .config
root@cnode1:~# make -j$(nproc) eapol_test

# (Optionnel) Rendre disponible globalement
root@cnode1:~# sudo cp eapol_test /usr/local/bin/
```

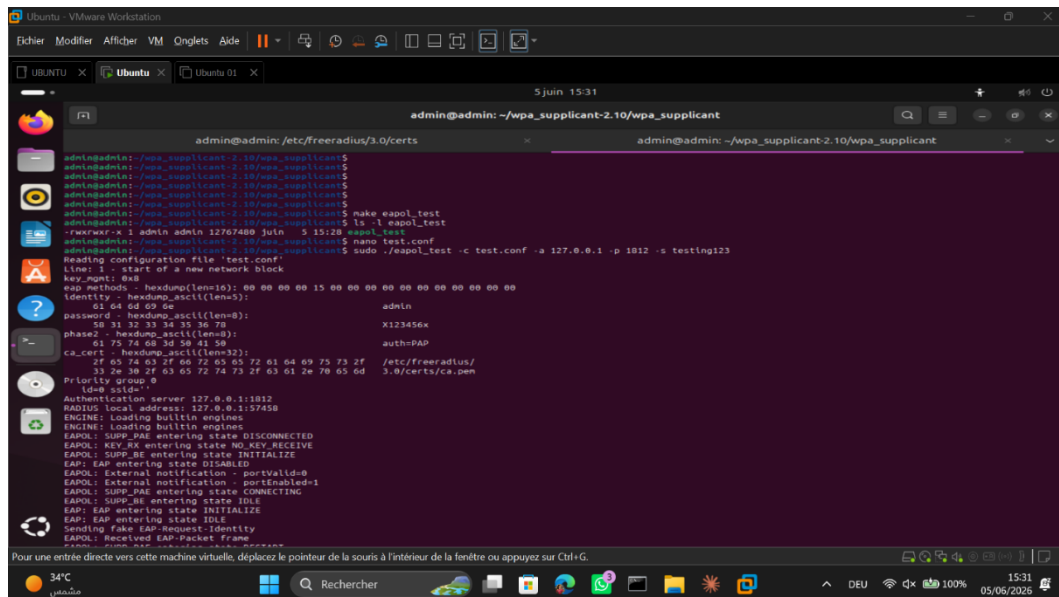


Figure 15 — Exécution du test d'authentification EAP-TTLS à l'aide de l'outil `eapol test`

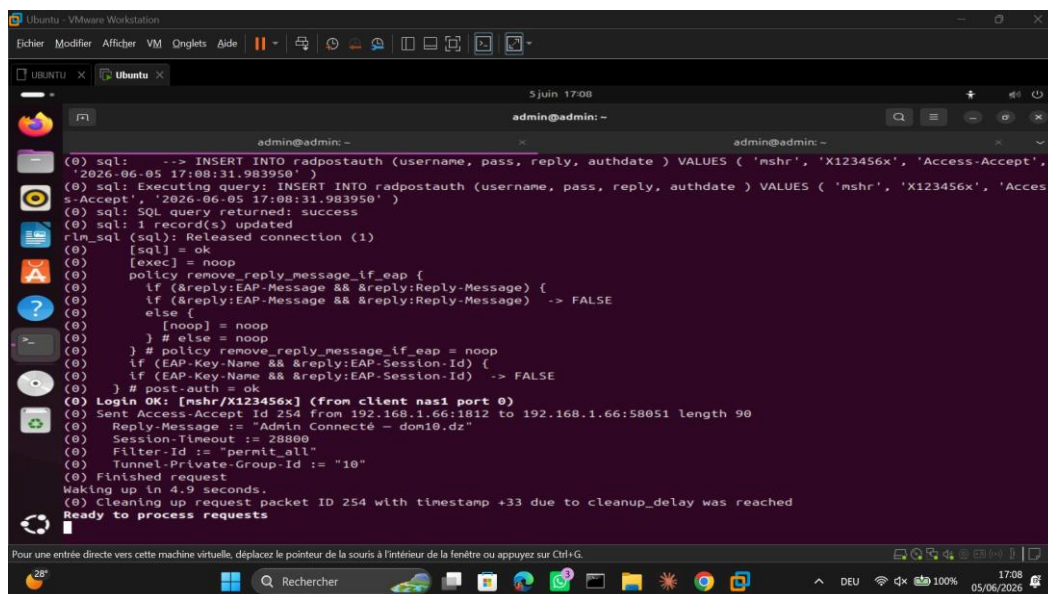


Figure 16— résultat d'une authentification EAP-TTLS réussie sur le serveur FreeRADIUS

6.4 — Copier le certificat CA et créer test.conf

```
# Copier ca.pem depuis le serveur vers le client
root@cnode1:~# sudo scp \
    root@192.168.1.66:/etc/freeradius/3.0/certs/ca.pem \
    ~/wpa supplicant-2.10/wpa supplicant/
```

```
# Créer le fichier de configuration du test EAP
root@cnode1:~/wpa_supplicant-2.10/wpa_supplicant/# nano test.conf

network={
    key_mgmt=IEEE8021X
    eap=TLS
    identity="mshr"
    password="X123456x"
    phase2="auth=PAP"
    ca_cert="ca.pem"
}
```

6.5 — Protocole de test — 2 terminaux simultanés

```
# Terminal 1 (serveur) - mode debug
root@serveur:~# sudo systemctl stop freeradius
root@serveur:~# sudo freeradius -X

# Terminal 2 (cnode1) - lancer eapol_test
root@cnode1:~/wpa_supplicant-2.10/wpa_supplicant/# sudo ./eapol_test \
    -c test.conf -a 192.168.1.66 -p 1812 -s testing123

# Résultat attendu - Terminal 2 :
EAP tunnel successfully established
SUCCESS

# Résultat attendu - Terminal 1 (freeradius -X) :
Auth-Type := PAP, Login OK
Sending Access-Accept

# Après les tests - toujours relancer le service normal !
root@serveur:~# sudo systemctl start freeradius
```

Erreur observée	Terminal	Cause probable	Solution
Access-Reject	Serveur	Utilisateur non trouvé dans LDAP ou mot de passe incorrect	Vérifier mshr dans phpLDAPadmin, mot de passe X123456x
TLS Alert	Client	Le client ne fait pas confiance au certificat du serveur	Vérifier que ca.pem dans test.conf est le bon CA FreeRADIUS
Timeout / No response	Client	Secret RADIUS incorrect ou port 1812 bloqué	Vérifier -s testing123 et sudo ss -uln grep 1812
Connection refused	Client	FreeRADIUS non démarré en mode -X	Vérifier que freeradius -X tourne dans Terminal 1

TP — Étape 7 : Tests, Débogage et Bonnes Pratiques

Un bon diagnostic suit toujours la même logique : vérifier d'abord que les services tournent, puis la connectivité réseau, puis les logs, et enfin les paquets. Cette section regroupe les commandes essentielles et les solutions aux erreurs les plus fréquentes rencontrées durant ce TP.

7.1 — Mode debug FreeRADIUS (-X)

C'est l'outil de diagnostic le plus puissant. Il affiche en temps réel chaque étape du traitement d'une requête : réception, recherche LDAP, évaluation des règles unlang, attributs insérés, réponse envoyée. Ne jamais laisser FreeRADIUS en mode -X en production : les mots de passe apparaissent en clair.

```
# Arrêter le service et lancer en mode debug
root@serveur:~# sudo systemctl stop freeradius
root@serveur:~# sudo freeradius -X

# Dans un autre terminal, générer un test
root@serveur:~# sudo radtest mshr X123456x 192.168.1.66 0 testing123

# Éléments clés à chercher dans la sortie -X :
# ✓ LDAP bind successful
# ✓ User found in LDAP
# ✓ LDAP-Group = admins
# ✓ Sending Access-Accept

# Tester uniquement la syntaxe (sans démarrer) :
root@serveur:~# sudo freeradius -XC
# Attendu : Configuration appears to be OK.
```

7.2 — Diagnostic LDAP

```
# Vérifier l'identité admin LDAP
root@serveur:~# sudo ldapwhoami \
    -H ldap://192.168.1.66 \
    -D "cn=admin,dc=dom10,dc=dz" -W
# → Saisir : X123456x

# Rechercher un utilisateur dans LDAP
root@serveur:~# sudo ldapsearch \
    -H ldap://192.168.1.66 \
    -D "cn=admin,dc=dom10,dc=dz" -W \
    -b "dc=dom10,dc=dz" "(uid=mshr)"

# Lister tous les groupes
root@serveur:~# sudo ldapsearch \
    -H ldap://192.168.1.66 \
    -D "cn=admin,dc=dom10,dc=dz" -W \
    -b "ou=groups,dc=dom10,dc=dz" "(objectClass=posixGroup)"

# Vérifier que slapd écoute sur 389
root@serveur:~# sudo ss -tlnp | grep 389
```

7.3 — Surveillance et capture réseau

```
# Sessions RADIUS actives
root@serveur:~# sudo radwho

# Logs FreeRADIUS en temps réel
root@serveur:~# sudo tail -f /var/log/freeradius/radius.log

# Logs accounting du jour
root@serveur:~# sudo tail -f \
    /var/log/freeradius/radacct/192.168.1.66/detail-$(date +%Y%m%d)

# Capture paquets RADIUS
root@serveur:~# sudo tcpdump -i ens33 -n port 1812 or port 1813

# Vérifier que FreeRADIUS écoute sur 1812 UDP
root@serveur:~# sudo ss -uln | grep 1812
```

7.4 — Tableau des erreurs courantes et solutions

Erreur	Cause	Solution
Failed to connect to LDAP	slapd arrêté ou port 389 bloqué	<code>sudo systemctl status slapd / ss -tlnp grep 389</code>
Access-Reject (mauvais MDP)	Mot de passe incorrect dans LDAP ou module LDAP non activé	Vérifier dans <code>phpLDAPadmin</code> + <code>ls mods-enabled/ldap</code>
No such attribute: LDAP-Group	Lien symbolique <code>mods-enabled/ldap</code> manquant	<code>sudo ln -s mods-available/ldap mods-enabled/ldap</code>
Secret does not match	Secret dans <code>clients.conf</code> différent du <code>radtest</code>	<code>sudo nano clients.conf</code> → <code>secret = testing123</code>
getent passwd mshr vide	SSSD offline ou <code>nsswitch.conf</code> sans 'sss'	<code>sudo sssctl domain-status default / journalctl -u sssd</code>
Permission denied — <code>sssd.conf</code>	Droits trop ouverts sur <code>/etc/sssd/sssd.conf</code>	<code>sudo chmod 600 sssd.conf && chown root:root sssd.conf</code>
FreeRADIUS refuse de démarrer	Erreur de configuration (certificat, port occupé)	<code>sudo freeradius -XC</code> pour voir la ligne d'erreur exacte
Fichier detail vide ou absent	Droits <code>freerad</code> manquants sur <code>/var/log/freeradius</code>	<code>sudo chown -R freerad:freerad /var/log/freeradius</code>
Attributs dupliqués dans <code>Accept</code>	Deux méthodes d'autorisation actives simultanément	Ne garder qu'unlang OU fichier <code>users</code> , jamais les deux

Conclusion de la Partie Pratique

Cette partie pratique a suivi une progression délibérément structurée, du plus simple vers le plus complexe, pour ancrer chaque concept dans une réalité opérationnelle concrète.

La Partie I a démontré que l'AAA local sur routeur Cisco est fonctionnel et simple à mettre en place, mais qu'il porte en lui les germes de ses limitations : chaque compte est une île, chaque équipement est une forteresse indépendante, sans cohérence ni traçabilité globale. La Partie II a montré comment le même routeur peut déléguer ses décisions à un serveur RADIUS externe, réalisant ainsi le premier niveau de centralisation. La configuration Cisco reste simple, mais le gain est immédiat : un seul point d'administration pour des dizaines d'équipements.

La Partie III, le travail pratique personnel de ce stage, a porté sur la construction de ce serveur central lui-même. De l'installation initiale de FreeRADIUS et SSSD jusqu'aux déclencheurs Zabbix en passant par les politiques unlang et l'authentification Wi-Fi EAP-TTLS, chaque étape a illustré l'un des piliers du modèle AAA : identifier (Authentication), contrôler (Authorization), tracer (Accounting).

BIBLIOGRAPHIE

NORMES ET STANDARDS (RFC / IEEE)

- [1] Rigney, C., Willens, S., Rubens, A., & Simpson, W. (2000). Remote Authentication Dial In User Service (RADIUS). Internet Engineering Task Force (IETF), RFC 2865. *Disponible sur :* <https://datatracker.ietf.org/doc/html/rfc2865>
- [2] Rigney, C. (2000). RADIUS Accounting. Internet Engineering Task Force (IETF), RFC 2866. *Disponible sur :* <https://datatracker.ietf.org/doc/html/rfc2866>
- [3] Carrel, D., & Grant, L. (1997). The TACACS+ Protocol (Version 1.78). Internet Engineering Task Force (IETF), RFC Draft (Cisco Systems).
Disponible sur : <https://datatracker.ietf.org/doc/html/draft-grant-tacacs-02>
- [4] Aboba, B., Blunk, L., Vollbrecht, J., Carlson, J., & Levkowetz, H. (2004). Extensible Authentication Protocol (EAP). Internet Engineering Task Force (IETF), RFC 3748. *Disponible sur :* <https://datatracker.ietf.org/doc/html/rfc3748>
- [5] IEEE Standards Association (2020). IEEE Standard for Local and Metropolitan Area Networks — Port-Based Network Access Control. IEEE Std 802.1X-2020. *Disponible sur :* <https://standards.ieee.org/ieee/802.1X/7345/>
- [6] Sermersheim, J. (2006). Lightweight Directory Access Protocol (LDAP): The Protocol. Internet Engineering Task Force (IETF), RFC 4511. *Disponible sur :* <https://datatracker.ietf.org/doc/html/rfc4511>
- [7] Simon, D., Aboba, B., & Hurst, R. (2008). The EAP-TLS Authentication Protocol. Internet Engineering Task Force (IETF), RFC 5216. *Disponible sur :* <https://datatracker.ietf.org/doc/html/rfc5216>
- [8] Palekar, A., Simon, D., Zorn, G., & Salowey, J. (2004). Protected EAP Protocol (PEAP) Version 2. Internet Engineering Task Force (IETF), Internet Draft. *Disponible sur :* <https://datatracker.ietf.org/doc/html/draft-josefsson-ppext-eap-tls-eap-10>

FREERADIUS — DOCUMENTATION OFFICIELLE

- [9] FreeRADIUS Project (2024). FreeRADIUS — The world's most popular RADIUS server: Official Documentation. FreeRADIUS.org. *Disponible sur :* <https://freeradius.org/documentation/>
- [10] FreeRADIUS Project (2024). FreeRADIUS Version 3 — Configuration Reference: modules/ldap. FreeRADIUS Documentation. *Disponible sur :* https://freeradius.org/radiusd/man/rlm_ldap.html
- [11] FreeRADIUS Project (2024). FreeRADIUS unlang — The FreeRADIUS Processing Language. FreeRADIUS Documentation. *Disponible sur :* <https://freeradius.org/radiusd/man/unlang.html>
- [12] FreeRADIUS Project (2024). EAP module configuration (rlm_eap) — EAP-PEAP, EAP-TLS, EAP-TTLS. FreeRADIUS Documentation. *Disponible sur :* <https://freeradius.org/documentation/freeradius-server/4.0/raddb/mods-available/eap.html>

OPENLDAP — DOCUMENTATION OFFICIELLE

- [13] The OpenLDAP Project (2024). OpenLDAP Software 2.6 Administrator's Guide. OpenLDAP Foundation. *Disponible sur :* <https://www.openldap.org/doc/admin26/>
- [14] The OpenLDAP Project (2024). OpenLDAP slapd Configuration Guide — SSSD Integration. OpenLDAP Foundation. *Disponible sur :* <https://www.openldap.org/doc/admin26/slapdconfig.html>
- [15] Red Hat, Inc. (2024). SSSD — System Security Services Daemon: Integration with LDAP. Red Hat Enterprise Linux 9 Documentation. *Disponible sur :* https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/9/html/integrating_rhel_systems_directly_with_windows_active_directory/index

CISCO SYSTEMS — DOCUMENTATION OFFICIELLE

- [16] Cisco Systems, Inc. (2024). Configuring Authentication — Cisco IOS Security Configuration Guide. Cisco Documentation (IOS 15.x). *Disponible sur :* https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/sec_usr_aaa/configuration/xr-16/sec_usr_aaa-xr-16-book/sec-cfg-authentication.html

- [17] Cisco Systems, Inc. (2024). RADIUS Configuration Guide — Configuring RADIUS (IOS). Cisco Documentation. Disponible sur : https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/sec_usr_rad/configuration/xe-16/sec-usr-rad-xe-16-book/sec-cfg-radius.html
- [18] Cisco Systems, Inc. (2024). AAA Command and Configuration Guide — aaa new-model, aaa authentication, aaa authorization, aaa accounting. Cisco IOS Security Command Reference. Disponible sur : <https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/security/a1/sec-a1-cr-book/sec-cr-a1.html>

ZABBIX — DOCUMENTATION OFFICIELLE

- [19] Zabbix LLC (2024). Zabbix 7.4 — Official Documentation: Zabbix Agent 2 Configuration. Zabbix Documentation. Disponible sur : https://www.zabbix.com/documentation/7.4/en/manual/appendix/config/zabbix_agent2
- [20] Zabbix LLC (2024). Zabbix 7.4 — User Parameters: Custom Metric Collection. Zabbix Documentation. Disponible sur : <https://www.zabbix.com/documentation/7.4/en/manual/config/items/userparameters>
- [21] Zabbix LLC (2024). Zabbix 7.4 — Triggers and Alerting Configuration. Zabbix Documentation. Disponible sur : <https://www.zabbix.com/documentation/7.4/en/manual/config/triggers>

SYSTEME D'EXPLOITATION — UBUNTU & SECURITE RESEAU

- [22] Canonical Ltd. (2024). Ubuntu Server Guide 24.04 LTS — Network Authentication and LDAP. Ubuntu Documentation. Disponible sur : <https://ubuntu.com/server/docs/service-ldap>
- [23] Canonical Ltd. (2024). UFW — Uncomplicated Firewall documentation (Ubuntu 24.04). Ubuntu Documentation. Disponible sur : <https://help.ubuntu.com/community/UFW>

OUVRAGES DE REFERENCE

- [24] Stallings, W. (2017). Network Security Essentials: Applications and Standards (6th ed.). Pearson Education. Chapitres 3 (Authentication), 4 (Access Control), 9 (Network Access Control & 802.1X).
- [25] Forouzan, B. A. (2013). Data Communications and Networking (5th ed.). McGraw-Hill. Chapitres sur les protocoles AAA, RADIUS, et la sécurité réseau.
- [26] RFC Editor (IETF) (2024). RFC Index — Authentication, Authorization, and Accounting (AAA) Protocols. IETF Datatracker. Disponible sur : <https://datatracker.ietf.org/wg/aaa/documents/>